

End-of-Year Project Report (PFA)

MODULE: DEVELOPMENT PROJECT

MAJOR: *ADVANCED SOFTWARE ENGINEERING FOR DIGITAL SERVICES (ASEDS)*

Design and Development of an Advanced LegalTech Platform using RAG to Answer Queries on Moroccan Law

Prepared by :

Qualid AMTOUT

Youness ESSABRI

Supervised by :

Pr. Zineb EL AKKAOUI (INPT)



AGENCE NATIONALE DE RÉGLEMENTATION DES TÉLÉCOMMUNICATIONS

INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS

Academic Year : 2025/2026

Dedicated to

Friends and family (1st paragraph)

Friends and family (2nd paragraph)

Your name.

Acknowledgments

Thank your supervisors (INPT and company), the managers and HR, the teachers at INPT and the jury.

Résumé

Avec l'évolution rapide des technologies d'Intelligence Artificielle, les modèles de langage ont prouvé une grande efficacité pour assimiler d'énormes quantités d'informations et les reformuler afin de répondre à des questions complexes. En revanche, les consultations juridiques traditionnelles et l'analyse de dossiers restent des processus très coûteux. Une utilisation appropriée de l'IA peut réduire ce coût de manière drastique pour le rendre presque symbolique.

Notre objectif à travers ce projet de fin d'année est de développer une plateforme numérique intégrée permettant aux utilisateurs d'obtenir des consultations juridiques précises, d'analyser leurs dossiers et de générer automatiquement des documents juridiques. Pour ce faire, nous avons d'abord mené une étude approfondie des besoins du marché et défini les fonctionnalités nécessaires pour résoudre les problématiques soulevées. Ensuite, nous avons modélisé l'ensemble du système à l'aide de diagrammes UML.

Nous avons été confrontés à un obstacle majeur : les modèles globaux (tels que Gemini, GPT, Claude) ne sont pas suffisamment entraînés sur le droit marocain, ce qui entraîne des « hallucinations » et des informations erronées. Pour y remédier, nous avons construit une architecture avancée basée sur la Génération Augmentée par la Recherche (RAG). Cela nous a permis de créer une base de données vectorielle avec ChromaDB, alimentée par la reconnaissance optique de caractères (OCR) via GPT-4o avec des optimisations spécifiquement adaptées aux documents marocains.

Sur le plan technique, nous avons utilisé FastAPI pour un backend performant, React pour les interfaces utilisateur, et PostgreSQL pour la gestion de la base de données relationnelle.

Mots clés : Consultations juridiques, Intelligence Artificielle, Hallucination, RAG, Base de données vectorielle, ChromaDB, GPT-4o, OCR, UML, FastAPI, React, PostgreSQL.

Summary

With the rapid evolution of Artificial Intelligence technologies, large language models have proven highly efficient in assimilating massive amounts of information and reformulating it to answer complex questions. Conversely, traditional legal consultations and case analyses remain highly expensive. Proper utilization of AI can drastically reduce this cost to a nominal fraction, making legal services more accessible.

Our goal through this end-of-year project is to develop an integrated digital platform that allows users to obtain accurate legal consultations, analyze their complex cases, and automatically generate legal documents. To achieve this, we first conducted a comprehensive market needs analysis and defined the core functionalities required to solve the target problems. We then modeled the entire system architecture using UML diagrams.

We faced a major obstacle: global models (like Gemini, GPT, and Claude) are not sufficiently trained on Moroccan law, leading to "hallucinations" and fabricated legal information. To solve this, we designed and built an advanced architecture based on Retrieval-Augmented Generation (RAG). This allowed us to construct a vector database using ChromaDB, populated via Optical Character Recognition (OCR) using GPT-4o with custom optimizations adapted to Moroccan documents.

Technically, we adopted FastAPI for a high-performance backend, React for dynamic user interfaces, and PostgreSQL for robust relational database management.

Key Words : Legal Consultations, Artificial Intelligence, Hallucination, RAG, Vector Database, ChromaDB, GPT-4o, OCR, UML, FastAPI, React, PostgreSQL.

ملخص

مع التطور المتسارع لتقنيات الذكاء الاصطناعي، أثبتت النماذج اللغوية قدرة وكفاءة عالية جداً في استيعاب المعلومات الضخمة وإعادة صياغتها للإجابة على التساؤلات المعقدة. في المقابل، لا تزال الاستشارات القانونية التقليدية وتحليل الملفات عملية مكلفة للغاية. في حين أن التوظيف السليم للذكاء الاصطناعي يمكن أن يقلص هذه التكلفة بشكل جذري لتصبح تكلفة شبه رمزية، مما يسهل ولوج الأفراد للخدمات القانونية.

هدفنا من خلال مشروع نهاية السنة (عذا) هو تطوير منصة رقمية متكاملة تتيح للمستخدمين الحصول على استشارات قانونية دقيقة، تحليل ملفاتهم المعقدة، وصياغة وتوليد الوثائق القانونية بشكل آلي. لتحقيق هذه الأهداف، قمنا بدايةً بدراسة احتياجات السوق وتحديد الوظائف الأساسية اللازمة لحل الإشكاليات المطروحة، ثم قمنا بنمذجة النظام وهيكلته الشاملة باستخدام مخططات أوبي.

واجهتنا عقبة جوهرية تتمثل في أن النماذج اللغوية العالمية المعروفة (مثل جمن و جعة و ثلود) لم يتم تدريبها بشكل كافٍ على النصوص التشريعية المغربية، مما يجعلها عرضة لظاهرة الهلوسة واختلاق معلومات خاطئة. لحل هذه المشكلة، قمنا بهندسة وبناء هيكلية متقدمة تعتمد على تقنية الاسترجاع المعزز بالتوليد (غاج). هذه الهيكلية مكنتنا من بناء قاعدة بيانات متجهة باستخدام تقنية شهرمض، معتمدين في تغذيتها على تقنية التعرف الضوئي على الحروف (اوثنغ) عبر نموذج جعة ٤١، مع إدخال تحسينات مصممة خصيصاً لتلائم الوثائق المغربية.

أما على مستوى البنية التحتية، فقد اعتمدنا على إطار العمل ذستاعي لتطوير الواجهة الخلفية، ومكتبة غأخت لبناء واجهات المستخدم، في حين استخدمنا نظام عستكرصئ لإدارة قواعد البيانات.

الكلمات المفتاحية: الاستشارات القانونية، الذكاء الاصطناعي، الهلوسة، غاج، قاعدة البيانات المتجهة، شهرمض، جعة ٤١، تقنية اوثنغ، مخططات أوبي، ذستاعي، غأخت، عستكرصئ.

List of Abbreviations

API	Application Programming Interface
AI	Artificial Intelligence
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
JWS	JSON Web Signature
LLM	Large Language Model
MVP	Minimum Viable Product
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VLM	Vision-Language Model

List of Figures

1.1	The evolution of Large Language Models.	2
1.2	Illustration of embedding vectors in a high-dimensional semantic space.	5
1.3	Hierarchical representation of the HNSW algorithm layers used in modern Vector Databases.	6
1.4	The standard Retrieval-Augmented Generation (RAG) architecture combining an LLM with an external vector retrieval mechanism.	8
2.1	Visual representation of the Scrum Agile methodology.	12
2.2	Example of team communication and daily stand-ups.	14
2.3	Version control trace highlighting Git commits and branches on GitHub.	15
3.1	Global Use Case Diagram of the ChatBot Juridique AI Platform.	22
3.2	Sequence Diagram illustrating Chat Session Management and Multimodal Interaction.	24
3.3	Sequence Diagram of the Retrieval-Augmented Generation (RAG) Pipeline execution.	25
3.4	Sequence Diagram of the Document Upload and AI-driven OCR Ingestion Process.	26
3.5	Sequence Diagram of AI-Assisted Document Generation and Hybrid Compilation.	27
3.6	UML Class Diagram representing the PostgreSQL Relational Database Schema.	28
4.1	React Logo	31
4.2	Vite Logo	31
4.3	Axios Logo	32
4.4	Clerk Logo	32
4.5	Lucide and Markdown Logos	33
4.6	FastAPI Logo	33
4.7	PostgreSQL Logo	34
4.8	SQLAlchemy Logo	34
4.9	ChromaDB Logo	35
4.10	Google Gemini Logo	35

4.11	OpenAI Logo	36
4.12	BAAI BGE-M3 Model	36
4.13	PyMuPDF Logo	37
4.14	LaTeX Compilation Tools	38
4.15	Postman Logo	39
4.16	Git Logo	39
4.17	GitHub Logo	39
5.1	Detailed Backend Component Architecture	42
5.2	RAG Pipeline Execution Architecture	44
6.1	Public Landing Page Outlining Core LegalTech Features.	50
6.2	Clerk Authentication Portals: Account Creation (Left) and Sign-In (Right).	51
6.3	Initial Chat Dashboard Layout and History Navigation Utilities.	51
6.4	Active RAG Chat Sessions Rendering Bilingual Markdown and Formatted Tables.	52
6.5	Live File Attachment Modal Triggered from the Chat Input.	53
6.6	DMS Interface Displaying the Virtual Folder Creation Modal.	53
6.7	DMS Explorer Highlighting Dynamic Sorting and Pagination Controls.	54
6.8	Initial State of the LaTeX Workspace and Project Explorer.	54
6.9	Split-Screen Editor Toggling Between Raw Code View (Top) and Visual PDF Render (Bottom).	55
6.10	AI Assistant Processing Document Context to Suggest Direct Edits.	56
6.11	Execution of the <code>/template</code> Slash-Command Autonomously Generating a Contract.	56

List of Tables

1.1	Comparison of Selected Models Based on Context Window, Cost, and Performance. . . .	3
6.1	FastAPI Endpoint Baseline Latency Metrics.	57
C.1	Table Example	66
C.2	Long table Example	66

Listings

C.1 Bash example	65
C.2 Python example	65

Contents

Dedication	i
Acknowledgments	ii
Résumé	iii
Summary	iv
Arabic Abstract	v
List of Abbreviations	vii
List of Figures	ix
List of Tables	x
List of Listings	xi
Table of Contents	xvi
1 State of the Art	1
1.1 Large Language Models	1
1.1.1 Overview	1
1.1.2 Advancements and Capabilities	2
1.1.3 Challenges and Opportunities	2
1.1.4 Conclusion	3
1.2 Vector Stores	4
1.2.1 Introduction	4
1.2.2 Embeddings and Their Importance	4

1.2.3	Memory and Efficiency Considerations	5
1.2.4	HNSW Algorithm in Vector Stores	5
1.2.5	Conclusion	6
1.3	RAG Architecture	7
1.3.1	Introduction	7
1.3.2	The Necessity of RAG Architecture	8
1.3.3	Conclusion	8
2	Project Context and Objectives	9
2.1	General Project Framework	9
2.1.1	General Context	9
2.1.2	Problem Statement	10
2.1.3	Project Objectives	10
2.2	Project Management and Methodology	12
2.2.1	Adopted Methodology	12
2.2.1.1	The Scrum Framework	12
2.2.1.2	Roles within Scrum	12
2.2.2	Role Distribution within the Team	13
2.2.3	Collaboration and Communication Tools	13
2.2.3.1	Instant Communication	13
2.2.3.2	Version Control and Source Management	14
3	Analysis and Specifications	16
3.1	Functional Requirements	17
3.1.1	User Interface (UI)	17
3.1.2	Large Language Model (LLM) Integration	17
3.1.3	Data Retrieval (RAG System)	18
3.1.4	Authentication and Authorization	18
3.1.5	System Monitoring and Reporting	18
3.2	Non-Functional Requirements	18
3.2.1	Performance	19
3.2.2	Scalability	19
3.2.3	Security	19

3.3	System Constraints	19
3.3.1	Budget Constraints	19
3.3.2	Time Constraints	20
3.3.3	Technological Constraints	20
3.4	System Modeling (UML)	20
3.4.1	Use Case Actors	20
3.4.2	Use Case Diagram	21
3.5	System Behavior (Sequence Diagrams)	23
3.5.1	Chat Management and Live File Upload	23
3.5.2	Core RAG Pipeline Execution	25
3.5.3	Document Upload and Data Ingestion	26
3.5.4	AI-Assisted Document Generation	26
3.6	Database Architecture and Optimization	28
3.6.1	Class Diagram (Entity-Relationship Model)	28
3.6.2	Database Optimization Strategies	28
4	Technology Overview and Justification	30
4.1	Frontend Technologies	31
4.1.1	React.js	31
4.1.2	Vite	31
4.1.3	Axios	32
4.1.4	Clerk	32
4.1.5	Lucide React & React Markdown	33
4.2	Backend Infrastructure	33
4.2.1	FastAPI	33
4.2.2	PostgreSQL	34
4.2.3	SQLAlchemy & Asyncpg	34
4.3	AI & Retrieval-Augmented Generation (RAG)	35
4.3.1	ChromaDB	35
4.3.2	Google Gemini API (Gemini 2.5 Flash)	35
4.3.3	OpenAI API (GPT-4o Vision)	36
4.3.4	Sentence-Transformers (BAAI/bge-m3)	36
4.3.5	FlagEmbedding (BGE-Reranker)	37

4.4	Utilities & Document Processing	37
4.4.1	PyMuPDF	37
4.4.2	LaTeXLite API & MiKTeX	37
4.4.3	uv (Python Package Manager)	38
4.5	Development and Collaboration Tools	38
4.5.1	Postman	38
4.5.2	Git	39
4.5.3	GitHub	39
5	Architecture Design	41
5.1	Detailed Component Architecture	42
5.1.1	Backend Design	42
5.2	RAG Architecture & Intelligence Pipeline	43
5.2.1	Phase 1: Offline Ingestion & Storage	43
5.2.2	Phase 2: Online Retrieval & Generation	43
5.3	Frontend Design	45
5.3.1	Application Routing Tree (Sitemap)	45
5.3.2	UI and Prototyping	45
6	Demonstration and Evaluation	48
6.1	User Expected Roadmap	49
6.2	Functional Workspace Demonstration	50
6.2.1	Portal Entry and Session Authentication	50
6.2.2	Interactive Consultation and RAG Navigation	51
6.2.3	Document Management System (DMS) Overview	53
6.2.4	LaTeX Legal Editor and AI Assistant	54
6.3	Technical and Algorithmic Evaluation	57
6.3.1	Backend API Performance and Latency Analysis	57
6.3.2	RAG Pipeline Accuracy and Retrieval Metrics	57
	General Conclusion and Perspectives	60
A	Glossary	62
B	Acronyms	63

C	Some subject you want to expand on	65
Bibliography		67

Chapter 1

State of the Art

Introduction

The rapid advancement of artificial intelligence, particularly in the development of Generative AI (GenAI) and Large Language Models (LLMs), has significantly impacted the field of intelligent systems. These technologies enable the generation of highly personalized and contextually relevant content, marking a pivotal shift in how AI can be applied across various domains.

A key innovation in this space is the Retrieval-Augmented Generation (RAG) architecture, which synergizes the generative capabilities of LLMs with advanced retrieval mechanisms to enhance the accuracy, relevance, and factual grounding of AI outputs. Central to the effectiveness of RAG are vector stores—specialized databases designed to store and retrieve high-dimensional vector embeddings, thereby enabling efficient semantic search and retrieval processes.

This chapter provides an in-depth exploration of these core technologies. It begins with the evolution and capabilities of LLMs, followed by the principles and applications of RAG architectures. Furthermore, it examines the critical role of vector stores in facilitating efficient information retrieval, establishing the theoretical foundation for the architectural decisions and innovations introduced in this project.

1.1 Large Language Models

1.1.1 Overview

Language is the fundamental medium for human communication and interaction with machines. The evolution of language models—from early statistical methods to neural approaches, and ultimately to modern LLMs—has revolutionized Natural Language Processing (NLP). The advent of transformer

architectures, coupled with enhanced computational capabilities and the availability of vast training corpora, has enabled LLMs to exhibit human-level performance across a myriad of language tasks [1, 2].

LLMs are uniquely capable of processing and generating coherent text, demonstrating remarkable generalization across multiple tasks, including translation, summarization, and open-domain question answering. These models have grown exponentially in both complexity and capability, transitioning from standard Pre-trained Language Models (PLMs) to massive LLMs that often comprise tens to hundreds of billions of parameters, trained on datasets spanning gigabytes to terabytes.

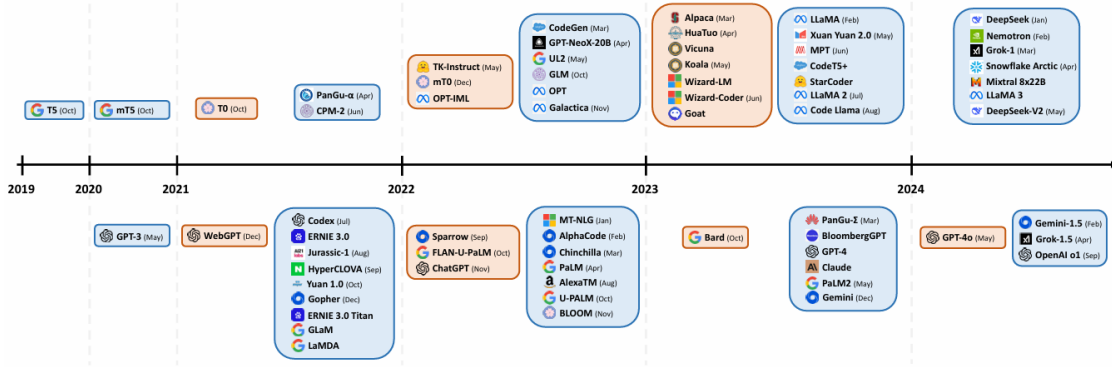


Figure 1.1: The evolution of Large Language Models.

1.1.2 Advancements and Capabilities

LLMs have demonstrated emergent abilities—such as zero-shot reasoning, logical decision-making, and in-context learning—which arise from their massive scale rather than explicit task-specific training. These capabilities allow LLMs to perform complex operations with minimal task-specific data, showcasing their profound generalization potential. Fine-tuning and alignment techniques (such as RLHF) further enhance their performance, enabling them to strictly follow user intent and generate highly accurate, safe responses.

Moreover, LLMs have shown significant improvements in multi-tasking and adapting to novel contexts. This versatility has positioned them as indispensable tools across diverse industries, ranging from healthcare to finance and legal technology. These advancements underscore the growing importance of LLMs as the cognitive engines of modern AI-driven applications.

1.1.3 Challenges and Opportunities

Despite their undeniable success, LLMs face significant operational challenges, including high computational costs, slow inference latency, and substantial hardware requirements. These limitations have spurred

intensive research into more efficient architectures, model compression techniques, and parameter-efficient fine-tuning strategies designed to make LLMs more accessible and practical for broader commercial applications. Methods such as model distillation, structured pruning, and weight quantization are actively being explored to reduce the immense resource intensity of these models.

One of the most critical challenges LLMs face is managing large-scale input data. While state-of-the-art LLMs can possess context windows of up to 2 million tokens, enterprise knowledge bases often span millions of documents, dwarfing these limits. This discrepancy restricts the ability of LLMs to process entire organizational datasets directly in a single inference step, complicating tasks that require simultaneous access to vast, dispersed information.

Additionally, the processing time for an LLM query is highly sensitive to the length of the input context. Longer queries demand significantly more computation because the self-attention mechanism processes each token, resulting in increased latency and higher financial costs. Consequently, handling extremely long contexts can become prohibitively expensive, particularly for applications requiring real-time responsiveness or frequent large-scale queries.

Furthermore, ensuring the ethical deployment of LLMs, mitigating algorithmic biases, and preventing the generation of misleading or harmful outputs remain critical concerns. Overcoming these hurdles presents rich opportunities for ongoing research and engineering innovation.

Model	Context Window	Cost (USD/1M Tokens)	Tokens/s	First Chunk (s)
GPT-4o	128k	\$7.50	97.4	0.45
GPT-4 Turbo	128k	\$15.00	31.9	0.60
GPT-3.5 Turbo	16k	\$0.75	88.6	0.38
Gemini 1.5 Pro	2M	\$5.25	64.6	1.05
Llama 3.1 405B	128k	\$4.50	29.4	0.66
Claude 3.5 Sonnet	200k	\$6.00	76.5	1.32
Mistral Large 2	128k	\$4.50	36.8	0.51
Mixtral 8x22B	65k	\$1.20	63.3	0.34

Table 1.1: Comparison of Selected Models Based on Context Window, Cost, and Performance.

1.1.4 Conclusion

LLMs represent a monumental advancement in Natural Language Processing, offering unprecedented capabilities in text comprehension and generation. However, the operational challenges associated with

their deployment—including computational efficiency, context window limitations, query processing latency, and ethical considerations—highlight the pressing need for continued innovation in model architecture and augmentation methodologies.

1.2 Vector Stores

1.2.1 Introduction

While LLMs have made significant strides, their inherent limitations in handling vast amounts of raw input data and long-tail queries remain a crucial bottleneck. Fortunately, emerging architectural paradigms are addressing this issue by providing highly sophisticated mechanisms for managing and accessing large-scale information dynamically during inference. These advancements promise to unlock the full potential of LLMs, enabling them to bypass strict token limits. In the following sections, we will explore vector storage—a groundbreaking architecture that fundamentally redefines large-scale data retrieval for generative AI models.

1.2.2 Embeddings and Their Importance

A vector store is a specialized database engineered to store and efficiently query dense vector embeddings, which are high-dimensional mathematical representations of unstructured data. These embeddings accurately capture the semantic meaning of complex information—such as text, images, or audio—and are paramount for applications requiring rapid, context-aware information retrieval based on semantic similarity rather than mere keyword matching.

Embeddings distill the semantic and syntactic content of data into a continuous vector space that machine learning models can process optimally. They empower systems to execute semantic searches by calculating the mathematical distance (e.g., cosine similarity) between vectors in high-dimensional space. Typically generated by deep neural networks such as BERT or BGE, these vectors encode the intricate relationships between distinct concepts.

Within this vector space, semantically similar concepts are positioned in close geometric proximity. For instance, synonyms like "contract" and "agreement" would yield embeddings located near one another, reflecting their shared contextual meaning. This capability is absolutely essential for applications demanding high-precision, contextually relevant information retrieval.

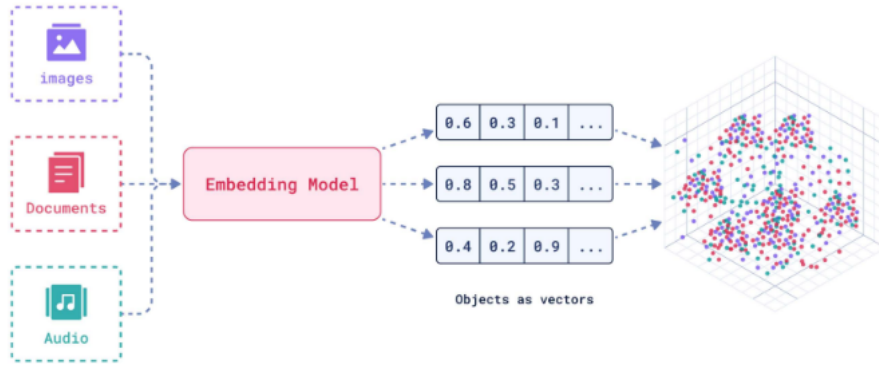


Figure 1.2: Illustration of embedding vectors in a high-dimensional semantic space.

1.2.3 Memory and Efficiency Considerations

The high-dimensional nature of vector embeddings—often consisting of hundreds or thousands of floating-point dimensions—can impose severe memory constraints on the hosting infrastructure. For example, a single 1024-dimensional vector stored as 32-bit floating-point numbers requires approximately 4 KB of memory. As an enterprise database scales into millions of vectors, the active memory demands become substantial.

To mitigate these memory overheads and accelerate retrieval speeds, sophisticated techniques such as Product Quantization or Binary Quantization are frequently employed. These approaches compress the vectors, drastically reducing their memory footprint while preserving sufficient topological information to guarantee effective retrieval. Converting a 1536-dimensional vector to a binary format, for instance, can reduce its memory requirement by a factor of 40, concurrently accelerating the dot-product calculations required for search. Despite this aggressive compression, high recall accuracy can be maintained by implementing a two-stage pipeline: utilizing compressed vectors for the initial rapid search, and full-precision vectors (or Cross-Encoders) for final ranking.

1.2.4 HNSW Algorithm in Vector Stores

The Hierarchical Navigable Small World (HNSW) algorithm is a state-of-the-art methodology for Approximate Nearest Neighbor (ANN) search, significantly augmenting the query performance of modern vector stores. Traditional brute-force search approaches possess a time complexity of $\mathcal{O}(N)$, rendering them highly impractical for massive datasets. HNSW successfully reduces this search complexity to $\mathcal{O}(\log N)$, enabling sub-millisecond retrieval times.

The HNSW algorithm organizes vector data into multiple hierarchical layers, each representing a

varying level of abstraction and connectivity. The foundational bottom layer contains all data points and is densely interconnected, whereas the upper layers contain exponentially fewer points and longer connections, forming a navigational hierarchy. During a search, the algorithm traverses the sparse upper layers to rapidly locate the general vicinity of the target. Once this approximate region is identified, the search cascades into the denser lower layers to pinpoint the exact nearest neighbors.

The spatial memory complexity of HNSW remains $\mathcal{O}(N)$, as each embedded point maintains a fixed maximum number of connections, ensuring memory consumption grows linearly rather than exponentially. This architectural elegance makes HNSW the industry standard for large-scale, high-velocity semantic retrieval.

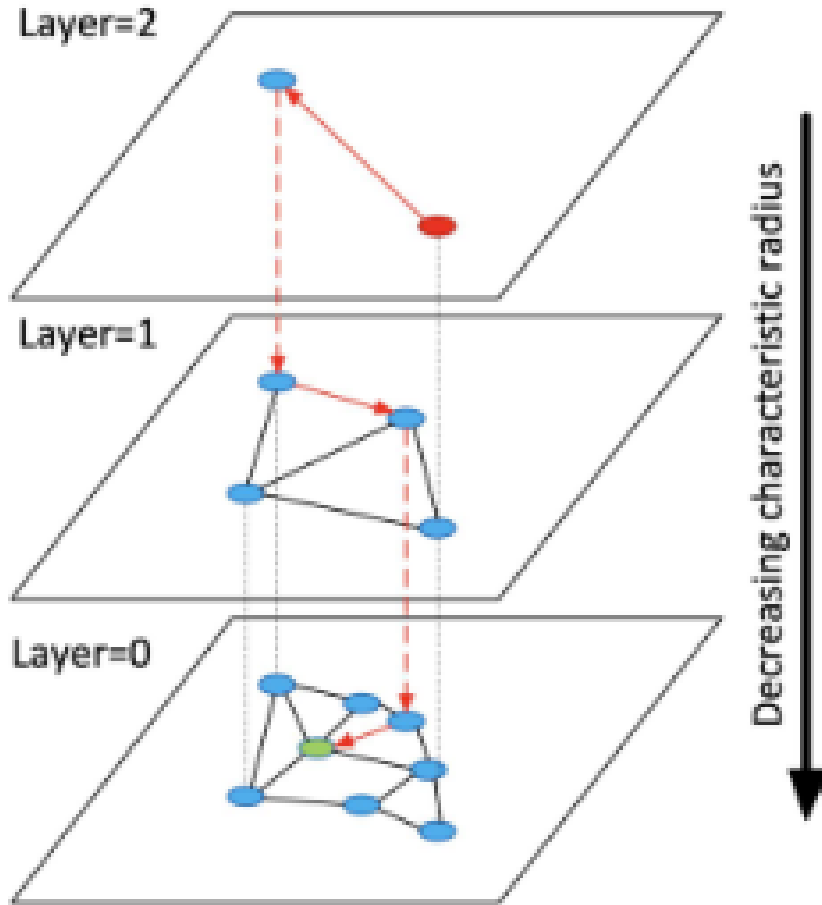


Figure 1.3: Hierarchical representation of the HNSW algorithm layers used in modern Vector Databases.

1.2.5 Conclusion

Vector stores, powered by indexing algorithms like HNSW, represent a paradigm shift in data retrieval technology. By leveraging dense high-dimensional embeddings and logarithmic search algorithms, vector

databases offer unparalleled performance for tasks requiring deep semantic understanding. While they demand more active memory than traditional relational databases, vector stores provide the critical speed and accuracy necessary to support next-generation AI applications, from enterprise semantic search to robust data analysis.

1.3 RAG Architecture

1.3.1 Introduction

Large pre-trained language models excel at internalizing vast quantities of factual knowledge within their parameters, achieving remarkable zero-shot performance across various NLP domains. However, they frequently struggle to access and manipulate this latent knowledge accurately, particularly in knowledge-intensive or highly specialized tasks (such as legal reasoning). This architectural limitation becomes glaringly evident when models hallucinate facts, fail to provide up-to-date information, lack the ability to cite provenance for their outputs, or require constant, expensive retraining to update their internal knowledge base.

Retrieval-Augmented Generation (RAG) elegantly resolves these fundamental challenges by integrating pre-trained parametric memory (the LLM) with dynamic, non-parametric memory (a vector database). In a RAG architecture, a generative model is actively fed by a neural retriever that searches a dense vector index representing a massive external corpus (e.g., millions of legal documents). This architecture empowers the system to dynamically fetch the most relevant, up-to-date information strictly during the inference generation process, resulting in highly specific, diverse, and empirically factual outputs.

By injecting a deterministic retrieval mechanism directly into the prompt pipeline, RAG models consistently demonstrate state-of-the-art performance in complex NLP tasks, dramatically outperforming parametric-only models while completely resolving the issues of knowledge provenance and hallucination.

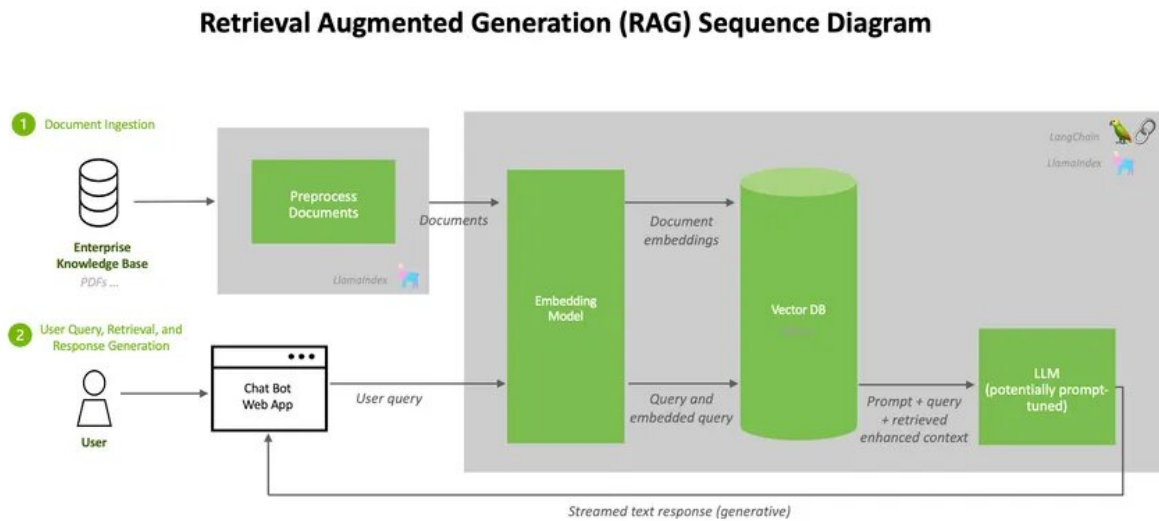


Figure 1.4: The standard Retrieval-Augmented Generation (RAG) architecture combining an LLM with an external vector retrieval mechanism.

1.3.2 The Necessity of RAG Architecture

Traditional language models rely exclusively on the static information encoded within their neural weights during the pre-training phase. While these models perform admirably on general conversational tasks, they fail critically on tasks demanding domain-specific, proprietary, or temporally recent information. An un-augmented model will frequently generate outdated or entirely fabricated responses because the knowledge it acquired during pre-training is inherently frozen in time and inherently generalized.

RAG architectures mitigate these limitations by decoupling knowledge storage from language generation. By integrating a dedicated retrieval mechanism, the model is permitted to query external, proprietary knowledge bases on-the-fly. This guarantees that the LLM is constantly grounded by accurate, contextually appropriate, and highly pertinent data fetched directly from a verified repository, effectively turning the LLM from a "memorizer" into an advanced "synthesizer" of factual information.

1.3.3 Conclusion

The RAG architecture, fundamentally underpinned by high-performance vector stores, constitutes a major breakthrough in the deployment of Enterprise AI. By fusing the reasoning capabilities of pre-trained language models with the deterministic accuracy of semantic retrieval mechanisms, RAG guarantees the generation of contextually aware and factually grounded outputs. Consequently, RAG serves as the definitive architecture for deploying secure, trustworthy AI applications in critical sectors such as law, medicine, and finance.

Chapter 2

Project Context and Objectives

Introduction

This chapter delineates the foundational context of the project, elucidates the specific challenges it seeks to address, and establishes the primary objectives that will guide the development of the proposed LegalTech platform.

2.1 General Project Framework

2.1.1 General Context

The contemporary global landscape is characterized by a rapid acceleration in digital transformation, presenting unprecedented opportunities to automate and optimize diverse sectors. However, the legal and administrative domain—particularly within Morocco—remains heavily reliant on traditional, manual methodologies. This pervasive reliance on legacy systems renders the acquisition of accurate legal information and the execution of administrative consultations inherently complex, financially burdensome, and highly resource-intensive.

As first-year engineering students enrolled in the "Advanced Software Engineering for Digital Services" (ASEDS) program at the National Institute of Posts and Telecommunications (INPT), we have cultivated a robust academic foundation encompassing software engineering principles, advanced database management, and the architecture of complex digital systems [2]. Capitalizing on this foundation and the ongoing revolution in artificial intelligence, we identified a compelling opportunity to harness these emerging technologies. By focusing specifically on the LegalTech domain, we aim to modernize this critical sector, facilitating seamless, reliable, and democratized access to legal information for the general public.

2.1.2 Problem Statement

Despite the pressing need to digitize the legal sector, the integration of artificial intelligence into the Moroccan legal framework encounters several distinct technical and economic hurdles. These challenges can be synthesized as follows:

- **Prohibitive Costs and Automation Deficits:** The systemic lack of automation in the legal sector renders professional consultations prohibitively expensive. Legal consultants typically mandate substantial fees, presenting the average user with a severe dilemma: either expend inordinate hours conducting manual, often fruitless web searches, or incur exorbitant costs for a fundamental legal inquiry.
- **Limitations of Global LLMs (The Hallucination Risk):** Despite the extraordinary capabilities of Large Language Models (LLMs), these architectures are predominantly trained on foreign, anglo-centric datasets and inherently lack deep, localized knowledge of Moroccan legal jurisprudence. Consequently, when queried regarding specific Moroccan legal procedures, these models frequently succumb to "hallucinations," generating confident but entirely fabricated and factually incorrect responses.
- **Data Extraction Bottlenecks (Unstructured Document Formats):** Moroccan legal documents and administrative circulars are rarely distributed in machine-readable, digital-native formats. Instead, they predominantly exist as scanned, image-based PDF files replete with complex tabular matrices, official stamps, and handwritten signatures. Traditional Optical Character Recognition (OCR) heuristics consistently fail to process and accurately extract structured semantic data from these artifacts.
- **Linguistic and Semantic Barriers:** The vast majority of official documentation is drafted in formal, highly structured Legal Arabic. Processing this specific dialect necessitates a robust, specialized Embedding Model explicitly trained on Arabic semantic structures to execute accurate mathematical vector searches.

2.1.3 Project Objectives

In direct response to these multi-faceted challenges, this project endeavors to architect and deploy an integrated digital platform. The successful realization of this platform is predicated on achieving the following primary and secondary objectives:

- Conduct a rigorous needs analysis and formally draft a comprehensive project specifications document.
- Evaluate and select an optimal, modern technology stack capable of supporting high-performance enterprise requirements.
- Architect an advanced Retrieval-Augmented Generation (RAG) system specifically designed to eradicate AI hallucination by forcing the generative model to synthesize answers exclusively from verified Moroccan legal sources.
- Implement state-of-the-art OCR pipelines and multilingual Embedding Models to ensure high-fidelity data ingestion.
- Develop an intuitive, responsive User Interface (UI) featuring:
 - A real-time chat interface allowing users to converse dynamically with the RAG-powered Legal LLM.
 - A multimodal bypass system empowering users to upload and query their personal documents directly.
 - A Split-Screen $\text{\LaTeX}$ Editor enabling users to autonomously draft, modify, and compile professional legal documents via code or through AI-driven template injection.
- Implement a robust Identity and Access Management (IAM) system for secure authentication.
- Enforce strict data security and Row-Level Security (RLS) policies to protect user metadata and uploaded artifacts.
- Design a highly normalized, asynchronous relational database.
- Engineer a comprehensive, JSON-based telemetry and logging system to monitor backend execution and AI latencies.
- Optimize backend concurrency to support high-volume parallel requests while minimizing operational cloud costs.
- Execute exhaustive Quality Assurance (QA) and end-to-end testing protocols to guarantee systemic stability.
- Finalize and execute the production deployment of the platform.

2.2 Project Management and Methodology

2.2.1 Adopted Methodology

2.2.1.1 The Scrum Framework

Scrum is an agile organizational framework utilized to manage the development of complex products. Defined by its creators as an iterative, holistic framework, Scrum focuses on achieving common objectives by delivering high-value products in a productive and creative manner. It is widely recognized as a premier implementation of the Agile Manifesto's core principles.

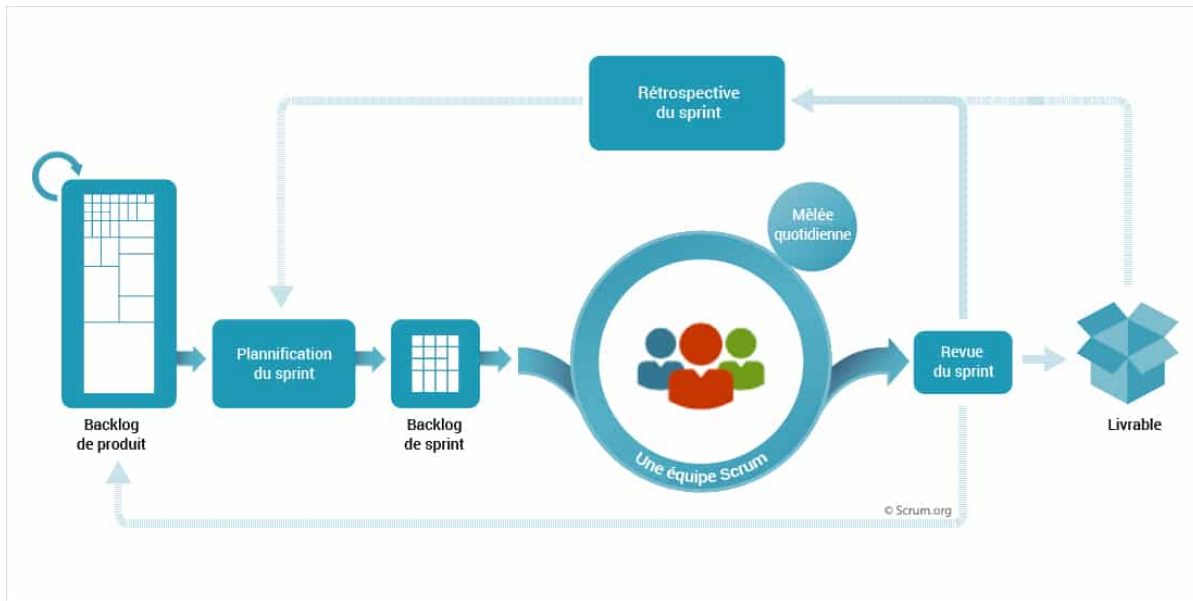


Figure 2.1: Visual representation of the Scrum Agile methodology.

Implementing a project under Scrum best practices requires the following foundational elements:

- An autonomous, highly responsible cross-functional team capable of rapidly adapting to dynamic client requirements or pivoting technical scopes.
- A structural workflow organized into "Sprints": short, time-boxed iterations (typically 2 to 4 weeks) facilitating rapid intervention and continuous integration.
- A rigorous definition of requirements, systematically consolidated into an evolving master task list known as the "Product Backlog."

2.2.1.2 Roles within Scrum

The Scrum framework prescribes three primary roles:

- 1) **The Product Owner:** The product's visionary and the primary proxy for the client's expectations. This role mandates high availability to resolve ambiguities and prioritize the backlog.
- 2) **The Scrum Master:** The facilitator responsible for ensuring absolute adherence to Scrum values. The Scrum Master shields the team from external disruptions and orchestrates the removal of developmental bottlenecks.
- 3) **The Development Team:** A self-managing, autonomous collective devoid of internal hierarchical structures, where technical decisions are deliberated and executed collaboratively.

These entities synergize to execute the following core Scrum events:

- **The Sprint:** A strict time-box defining the developmental cycle.
- **Sprint Planning:** The initial event where the team collaboratively selects specific User Stories from the Product Backlog to target during the impending Sprint.
- **Sprint Execution:** The active developmental phase, encompassing architectural design, programming, and testing.
- **Sprint Review and Retrospective:** The concluding evaluation where the team demonstrates the newly implemented features and introspects on workflow improvements for subsequent Sprints.
- **Backlog Grooming:** The continuous refinement and re-prioritization of the Product Backlog to accommodate shifting project dynamics.

2.2.2 Role Distribution within the Team

Given the streamlined nature of our academic development team for this project, the allocation of a dedicated, full-time Scrum Master was structurally impractical. Consequently, we adopted an adapted, highly collaborative Scrum paradigm, dynamically distributing the responsibilities of the Product Owner, Scrum Master, and Developers amongst ourselves to best simulate enterprise Agile workflows.

2.2.3 Collaboration and Communication Tools

2.2.3.1 Instant Communication

We utilized direct messaging platforms (such as WhatsApp) as our primary conduit for rapid coordination, conducting four distinct types of Agile meetings:

- **Daily Stand-ups:** Brief syncs allowing team members to report progress, highlight immediate goals, and flag technical blockers.
- **Pair-Programming Syncs:** Technical deep-dives utilized for complex architectural design and collaborative debugging sessions.
- **Product Demonstration Reviews:** Milestone meetings dedicated to showcasing compiled features and reviewing the project's holistic functionality prior to final delivery.

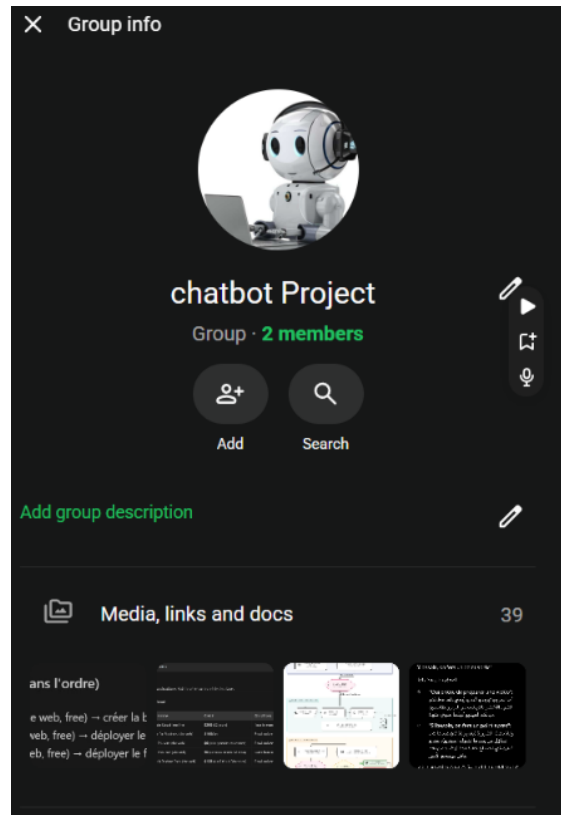


Figure 2.2: Example of team communication and daily stand-ups.

2.2.3.2 Version Control and Source Management

We strictly enforced version control methodologies utilizing **Git** and **GitHub**. This centralized approach guaranteed that the entire team maintained synchronized access to the latest verified codebase.

We adopted a strict **Branching and Merging** strategy, ensuring features were isolated during development and only merged into the main production branch following rigorous peer review. Furthermore, every **Commit** was carefully documented with descriptive syntax, ensuring absolute traceability of the platform's evolutionary history, as illustrated below.

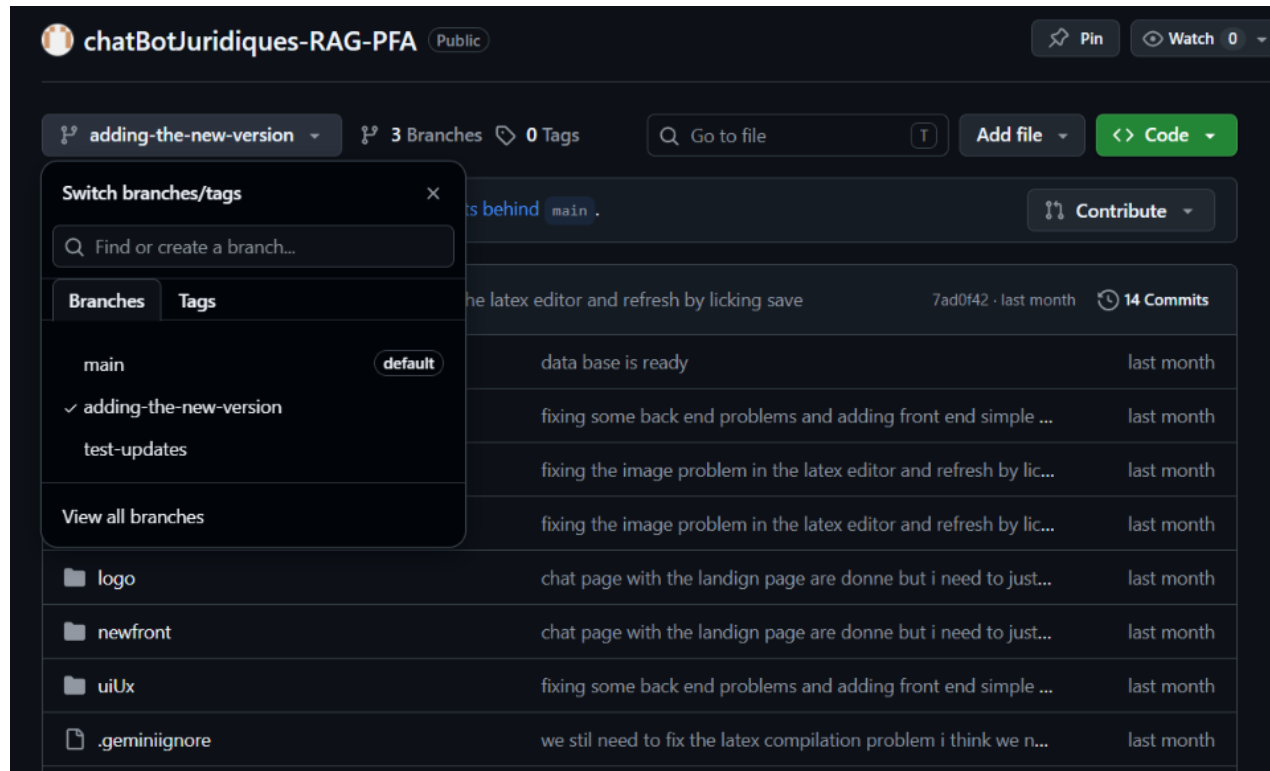


Figure 2.3: Version control trace highlighting Git commits and branches on GitHub.

Conclusion

Having firmly established the systemic context, diagnosed the core industry bottlenecks, and articulated the definitive objectives required for project success, the theoretical framework of the LegalTech platform is complete. The subsequent chapters will detail the precise needs specification, the system modeling, and the highly optimized architectural design required to materialize this vision.

Chapter 3

Analysis and Specifications

Introduction

This chapter is dedicated to delineating the detailed specifications and requirements of the proposed LegalTech platform. A rigorous analysis is essential to ensure the system architecture aligns with both the overarching project objectives and the specific expectations of end-users. In the subsequent sections, we categorize these specifications into functional requirements, detailing the explicit capabilities the system must exhibit; non-functional requirements, outlining the performance and operational standards; and system constraints, which establish the developmental, financial, and technological boundaries.

3.1 Functional Requirements

This section enumerates the functional requirements foundational to the platform’s architecture. These represent the core features and behaviors the system must support to facilitate a robust, AI-powered legal application.

3.1.1 User Interface (UI)

- **Landing Page:** The platform shall provide an intuitive, user-centric landing page that clearly articulates the application’s core concept, value proposition, and primary functionalities.
- **Account Creation & Login:** Users must be empowered to create an account and authenticate securely utilizing multiple Identity Providers (IdPs), including Email/Password, Google, Facebook, and Apple ID.
- **AI Chat Interface:** A dedicated conversational interface must be provisioned, allowing users to interact directly with the Large Language Model (LLM) augmented by the RAG pipeline.
- **File Upload & Management:** Users must be able to upload personal documents directly into the chat interface. Additionally, they require a persistent, personal cloud repository to store documents and import them into active chat sessions on demand.
- **Document Generation:** The platform shall enable users to draft formal legal documents manually using $\text{\LaTeX}$ via direct code input within an integrated IDE.
- **AI-Assisted Drafting:** Users shall have the ability to prompt the AI assistant to autonomously draft, format, or modify legal documents utilizing a centralized registry of pre-existing templates.

3.1.2 Large Language Model (LLM) Integration

- **Model Selection:** The platform must seamlessly integrate a highly capable, state-of-the-art LLM optimized for complex legal reasoning and multilingual comprehension.
- **Context Memory:** The conversational agent must retain conversational context, effectively remembering previous prompts and uploaded documents within a contiguous session.

- **Role Prompting:** The LLM must be strictly governed by engineered system prompts, enforcing rigorous rules and constraints to ensure it consistently operates within its designated persona as a Moroccan legal assistant.

3.1.3 Data Retrieval (RAG System)

- **RAG Architecture:** A Retrieval-Augmented Generation (RAG) paradigm shall be implemented to synthesize LLM-generated responses with highly relevant, empirically factual data extracted from a curated document corpus.
- **Standardized Retrieval Mechanism:** The system shall leverage specialized Vector Stores (ChromaDB) to facilitate rapid, efficient, and semantically precise searches based on user queries.

3.1.4 Authentication and Authorization

- **Multi-Provider Auth:** The system must implement robust, enterprise-grade authentication by integrating third-party OAuth providers via Clerk.
- **Admin Dashboard:** A secure, restricted-access dashboard must be exclusively available to platform administrators for user governance and systemic monitoring.

3.1.5 System Monitoring and Reporting

- **Smart Logging System:** An automated, highly structured telemetry mechanism must be implemented. To optimize disk storage, logs shall be compressed into `.gz` archives daily and subjected to a 30-day rolling deletion policy.
- **Automated API Retry Mechanism:** To minimize error rates and ensure high availability, backend controllers must autonomously retry transiently failed API requests prior to propagating errors to the client.

3.2 Non-Functional Requirements

This section outlines the non-functional requirements that dictate the operational behavior of the architecture, focusing heavily on critical attributes such as performance, scalability, and security.

3.2.1 Performance

- **Concurrency:** The application infrastructure must efficiently handle up to 100 concurrent user sessions without experiencing degradation in service quality.
- **Response Time:** The RAG pipeline and LLM inference engine must deliver responses to complex queries within a maximum threshold of 120 seconds.
- **Input Handling:** The architecture must be optimized to process massive input payloads (e.g., multimodal prompts or raw document text exceeding 10,000 characters) without catastrophic performance bottlenecks.

3.2.2 Scalability

- **Horizontal Scaling:** The architecture shall be intrinsically scalable, permitting the platform to dynamically augment capacity by spinning up additional instances of core components (e.g., FastAPI workers, vector databases).

3.2.3 Security

- **Data Protection:** The platform must enforce industry-standard encryption protocols (TLS/SSL) for data in transit and secure persistence mechanisms for data at rest.
- **Vulnerability Mitigation:** The application infrastructure must be rigorously protected against pervasive web vulnerabilities, specifically Distributed Denial of Service (DDoS) attacks and SQL Injection.

3.3 System Constraints

Constraints delineate the mandatory boundaries and operational limitations within which the project must be engineered and maintained.

3.3.1 Budget Constraints

- **Cost Optimization:** The architecture must be meticulously optimized for financial efficiency, minimizing long-term operational expenditures related to API token usage and cloud hosting.

- **Flexible Deployment:** The architecture must be containerized or highly portable to support flexible deployment options, facilitating rapid migration to cost-effective hosting providers if economically necessary.

3.3.2 Time Constraints

- **Development Timeline:** While the underlying architecture is designed for long-term extensibility, the complete development, testing, and deployment cycle of this iteration must strictly adhere to a 3-month academic project timeline.

3.3.3 Technological Constraints

- **Pre-Approved Tech Stack:** The system architecture must strictly conform to a pre-approved technology stack: **Python** (FastAPI) for all backend services and AI orchestration, and **React** (Vite) for all frontend client development.
- **Stack Deviations:** Any deviation from or addendum to the core technology stack must be rigorously justified and formally approved by project stakeholders.

3.4 System Modeling (UML)

3.4.1 Use Case Actors

An actor specifies a role played by a user or any external system that interacts directly with the subject. Actors are always external to the system boundaries, initiating use cases, providing inputs, or receiving outputs.

Within our platform ecosystem, the following actors have been identified:

Human Actors

- **User:** The primary end-user of the platform. This actor manages their profile, interacts with the AI chatbot, uploads legal documents, and compiles $\text{\LaTeX}$ templates into final PDF artifacts.
- **Administrator:** A privileged user entrusted with the overarching governance of the platform. They possess the authority to oversee, control, and audit registered users and their respective access rights.

- **Log Responsible:** A specialized technical role dedicated to system maintenance. They are responsible for monitoring system telemetry, tracking runtime errors, and managing data retention policies to guarantee platform stability.

External Systems (External Actors)

- **Clerk Auth:** An external Identity and Access Management (IAM) provider invoked by the platform to securely broker user registration, authentication, and session generation.
- **Gemini LLM:** The external Large Language Model API that functions as the cognitive engine of the platform, executing context-aware legal reasoning and text generation.
- **GPT-4o OCR:** An external, AI-driven Vision model integrated specifically to perform high-fidelity Optical Character Recognition (OCR), extracting semantic text and Markdown tables from complex scanned documents.
- **LaTeXLite:** An external compilation engine service utilized by the workspace to seamlessly transpile user-generated $\text{\LaTeX}$ code into rendered PDF documents.

3.4.2 Use Case Diagram

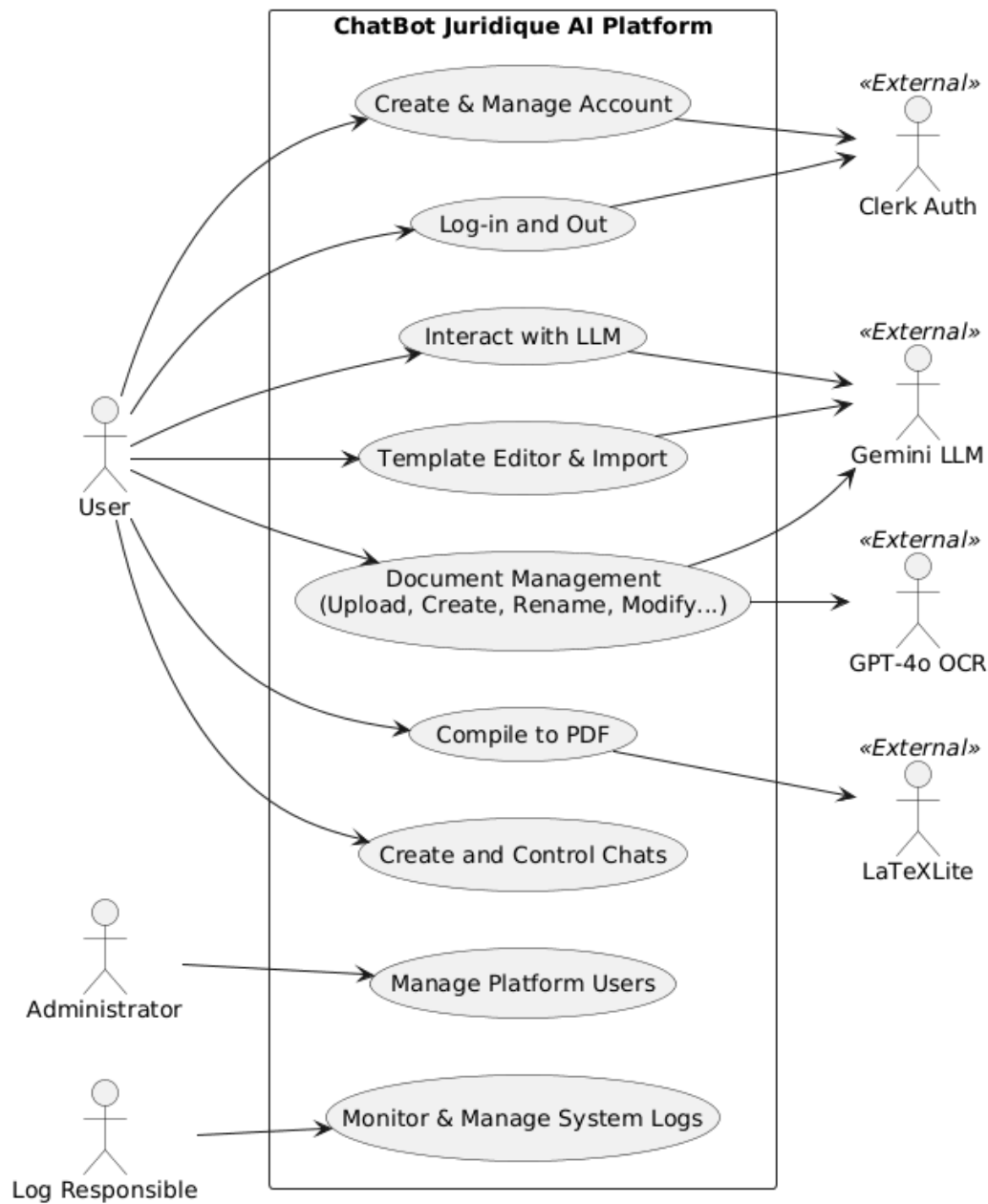


Figure 3.1: Global Use Case Diagram of the ChatBot Juridique AI Platform.

3.5 System Behavior (Sequence Diagrams)

Sequence diagrams are imperative for visualizing the dynamic, temporal behavior of the system architecture. They meticulously map how internal components, backend APIs, and external services interact sequentially to execute the core functionalities of the platform.

3.5.1 Chat Management and Live File Upload

This sequence diagram comprehensively details the lifecycle of a chat session. It illustrates the workflow when a user initializes a new session, optionally uploads a live file (triggering a multimodal prompt bypass), transmits prompts to the RAG pipeline, and modifies chat metadata (e.g., renaming or pinning) via the FastAPI backend and PostgreSQL database.

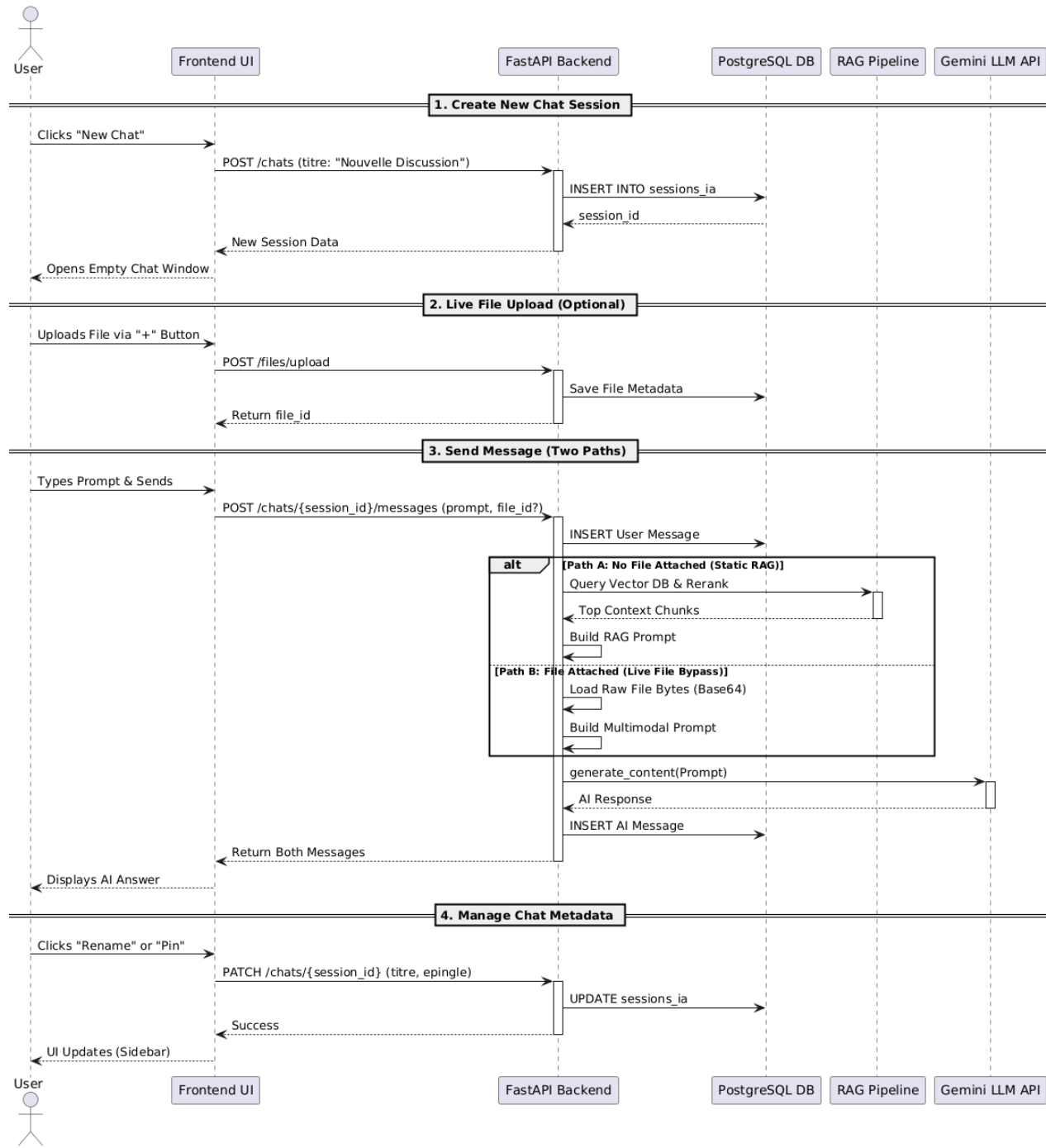


Figure 3.2: Sequence Diagram illustrating Chat Session Management and Multimodal Interaction.

3.5.2 Core RAG Pipeline Execution

This diagram maps the sophisticated query execution process when a user poses a legal question. It tracks the trajectory of the user's prompt as it is mathematically embedded, retrieved via cosine similarity in ChromaDB, rigorously reranked for maximum contextual relevance, and ultimately transmitted to the Gemini LLM to generate a factually grounded response.

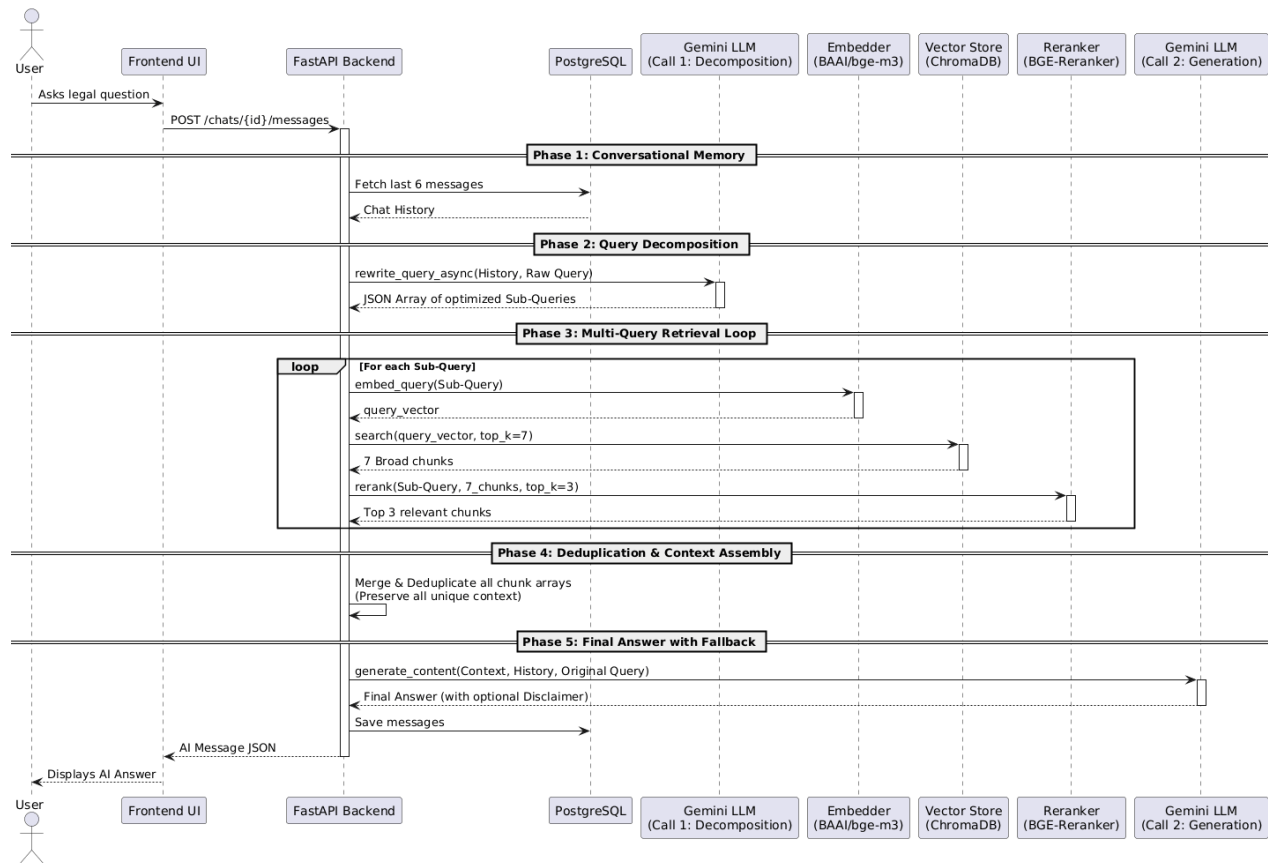


Figure 3.3: Sequence Diagram of the Retrieval-Augmented Generation (RAG) Pipeline execution.

3.5.3 Document Upload and Data Ingestion

This workflow details the highly complex offline data ingestion pipeline designed for Moroccan legal documents. It maps how raw PDFs are localized, converted into base64 images, processed by the GPT-4o Vision OCR to strip visual noise and extract semantic data, parsed by a custom XML chunker, and finally indexed into the vector database.

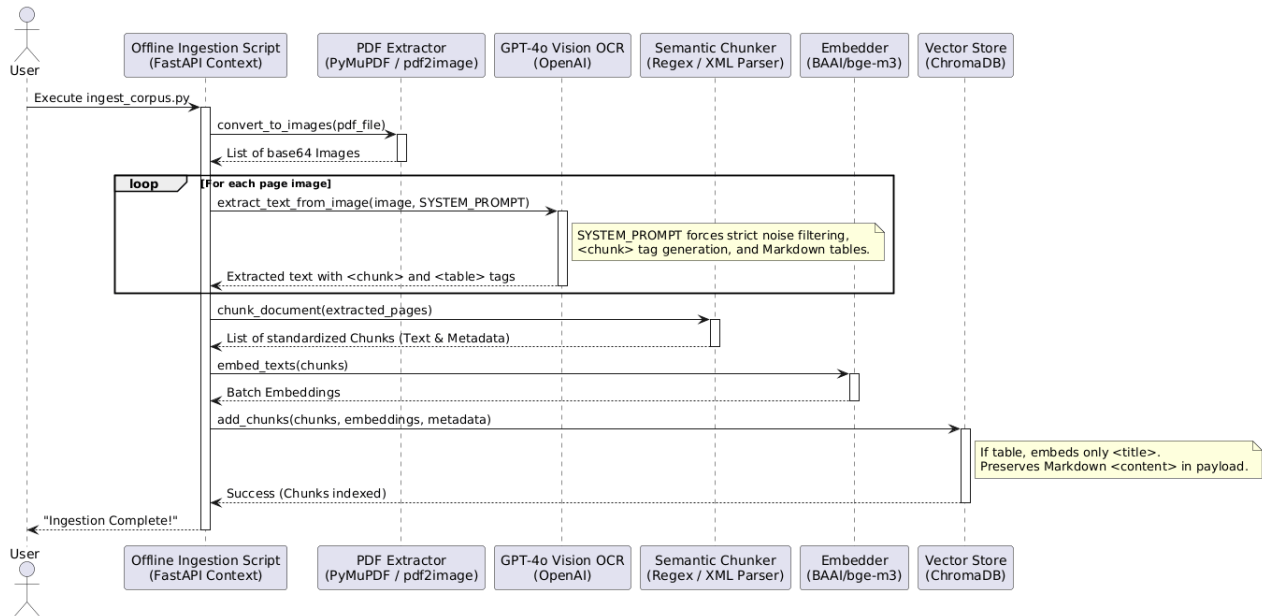


Figure 3.4: Sequence Diagram of the Document Upload and AI-driven OCR Ingestion Process.

3.5.4 AI-Assisted Document Generation

This sequence diagram illustrates the $\text{\LaTeX}$ document drafting and compilation feature. It maps how slash-commands (e.g., `/template`) trigger JSON template retrieval, which the LLM then adapts into syntactically valid $\text{\LaTeX}$ code. Subsequently, it traces the dual-path compilation request—dynamically selecting between a local pdfLaTeX engine and the LaTeXLite cloud API—to render the final PDF.

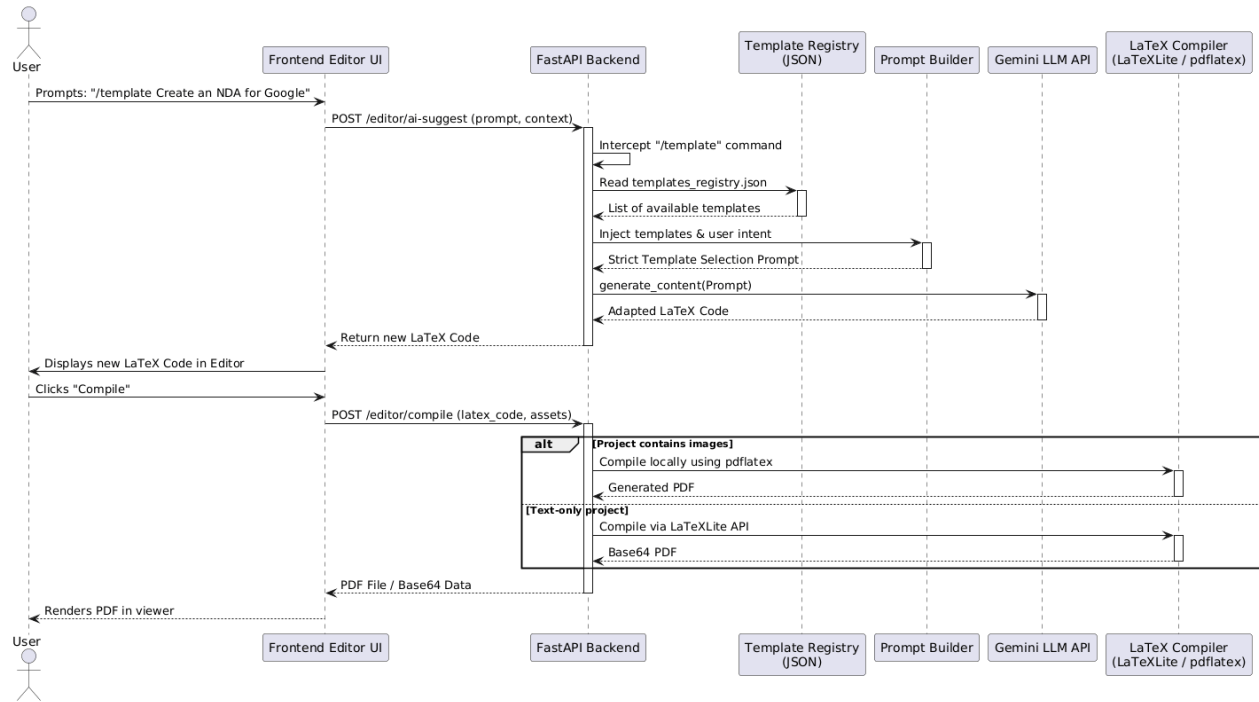


Figure 3.5: Sequence Diagram of AI-Assisted Document Generation and Hybrid Compilation.

3.6 Database Architecture and Optimization

3.6.1 Class Diagram (Entity-Relationship Model)

The Class Diagram serves as the foundational architectural blueprint for the PostgreSQL database. It illustrates the core entities—such as Users, Chat Sessions, Messages, and Documents—along with their precise attributes, primary keys, foreign keys, and relational multiplicities, ensuring a highly normalized and structurally sound data model.

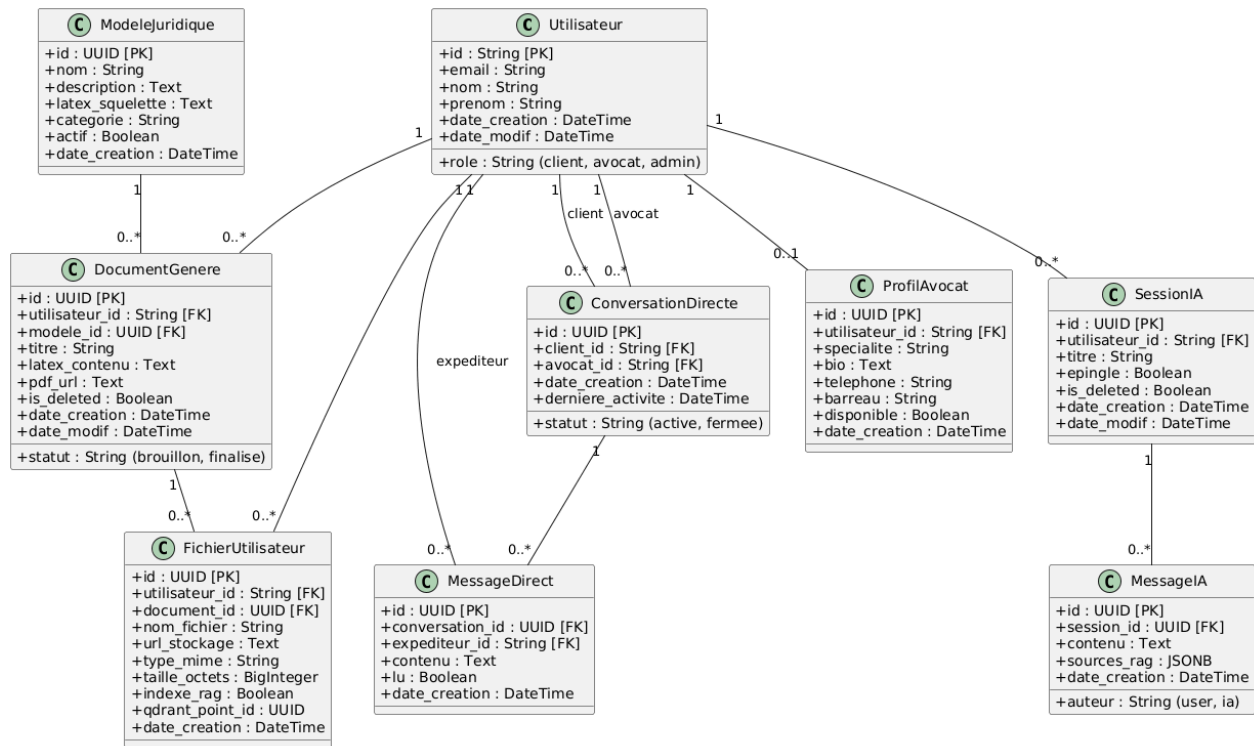


Figure 3.6: UML Class Diagram representing the PostgreSQL Relational Database Schema.

3.6.2 Database Optimization Strategies

To guarantee enterprise-grade performance, absolute data integrity, and secure tenant isolation, advanced structural optimizations were implemented directly within the PostgreSQL engine.

- **Advanced Indexing Strategies:** To minimize latency on high-volume queries, several complex index structures were deployed:
 - *Partial & Compound Indexes:* Deployed `idx_sessions_active` to index strictly active sessions while pre-sorting by `date_modif DESC`, enabling instantaneous UI rendering.

- *Foreign Key Indexing*: Explicitly indexed all relational foreign keys to completely eliminate full-table scans during heavy JOIN operations.
- *GIN Indexing*: Applied Generalized Inverted Indexes (GIN) to RAG metadata, facilitating rapid semantic searches within JSONB payloads without requiring deserialization.
- **PL/pgSQL Triggers & Automation**: A custom PL/pgSQL function coupled with BEFORE UPDATE triggers autonomously manages all operational timestamps (e.g., dynamically setting `date_modif` to `now()`). This ensures absolute chronological integrity and significantly reduces application-layer boilerplate logic.
- **Data Integrity & Constraints**: Rigorous relational constraints prevent orphaned data anomalies and corrupted system states:
 - *Cascading Deletes (ON DELETE CASCADE)*: All child dependencies are strictly bound to parent identifiers, ensuring pristine deletions in a single, atomic transaction.
 - *Check Constraints (CHECK)*: Constrained vocabularies (e.g., user roles and document statuses) are strictly enforced directly on columns, rejecting malformed strings before they can corrupt the database.
- **Aggregated Views**: A materialized SQL View (`v_dashboard_utilisateur`) was engineered to pre-calculate complex, heavy aggregations (such as session volumes, document counts, and RAG ingestion stats) in a single pass. This unlocks instantaneous analytics retrieval for the Dashboard API without incurring the severe performance penalties of real-time COUNT operations.

Conclusion

This chapter has established a comprehensive blueprint for the platform by systematically detailing the functional, non-functional, and systemic requirements. By explicitly outlining the user interface paradigms, RAG and LLM integrations, strict performance benchmarks, and robust database optimizations, we have defined the exact technical criteria necessary for a successful implementation. These foundational specifications serve as the definitive reference guiding the software development phases discussed in subsequent chapters.

Chapter 4

Technology Overview and Justification

Introduction

This chapter provides an in-depth overview of the frontend, backend, and AI technologies chosen for the development of our GenAI-powered LegalTech system. Each tool and framework was meticulously selected based on its performance, scalability, ease of integration, and ability to meet the project's specific architectural requirements. The following sections explore the functional overview of these technologies and the rationale behind their implementation in our architecture, from interactive user interfaces to advanced Retrieval-Augmented Generation (RAG) pipelines.

4.1 Frontend Technologies

4.1.1 React.js

React.js is a declarative, component-based JavaScript library maintained by Meta, used extensively for building highly interactive user interfaces and Single Page Applications (SPAs).

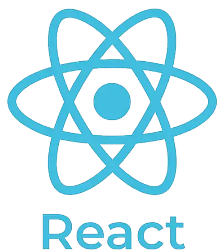


Figure 4.1: React Logo

Why it was used: React was chosen to build the dynamic, modular interfaces of the platform—specifically the split-screen LaTeX editor, the real-time chat interface, and the interactive document management system. Its component-based state management ensures a smooth, application-like user experience without disruptive page reloads.

4.1.2 Vite

Vite is a modern frontend build tool and development server that provides instantaneous Hot Module Replacement (HMR) and highly optimized production builds.



Figure 4.2: Vite Logo

Why it was used: It drastically accelerates frontend development time compared to older bundlers (like Webpack). Vite ensures that the final deployed application is minified, highly performant, and lightweight, which is critical for modern web applications.

4.1.3 Axios

Axios is a robust, promise-based HTTP client for JavaScript that operates seamlessly within the browser. It provides a clean, easily configurable API for executing asynchronous network requests and handling responses.



Figure 4.3: Axios Logo

Why it was used: Axios was selected to serve as the critical communication bridge between the React frontend and the FastAPI backend. It was specifically implemented to handle asynchronous REST API calls, automatically manage JSON data transformation, and utilize request interceptors to securely inject Clerk authentication tokens into HTTP headers. This ensures that all data exchange—from real-time chat prompts to heavy document uploads—remains secure, stable, and highly reliable across the platform.

4.1.4 Clerk

Clerk is a comprehensive Identity and Access Management (IAM) platform built for modern web applications, providing drop-in authentication components and secure session management.



Figure 4.4: Clerk Logo

Why it was used: Clerk was implemented to guarantee enterprise-grade security for user accounts. It handles login, registration, and token generation effortlessly, ensuring that sensitive legal data, chat histories, and uploaded files are strictly isolated and protected per user.

4.1.5 Lucide React & React Markdown

Lucide is a clean, modern iconography library, while React Markdown is a component used to safely parse and render Markdown syntax directly into HTML elements.



Figure 4.5: Lucide and Markdown Logos

Why it was used: Lucide provides the professional visual cues (folder icons, chat icons) across the platform. React Markdown is absolutely critical for rendering the raw mathematical tables, bold text, and structured lists generated by the AI into a highly readable, Notion-style chat interface.

4.2 Backend Infrastructure

4.2.1 FastAPI

FastAPI is a modern, incredibly fast web framework for building APIs with Python. It is based on standard Python type hints and native asynchronous programming.



Figure 4.6: FastAPI Logo

Why it was used: Selected as the core backend engine due to its exceptional performance and native asynchronous (`async/await`) capabilities. This is vital for a platform that must simultaneously juggle non-blocking database queries and long-running AI generation requests without freezing the server.

4.2.2 PostgreSQL

PostgreSQL is an advanced, open-source object-relational database system globally recognized for its robust feature set, reliability, and strict data integrity constraints.



Figure 4.7: PostgreSQL Logo

Why it was used: PostgreSQL acts as the absolute source of truth for the platform. It reliably stores, links, and indexes all relational metadata, including user profiles, session histories, chat logs, virtual folder hierarchies, and LaTeX templates.

4.2.3 SQLAlchemy & Asyncpg

SQLAlchemy is a comprehensive Python SQL toolkit and Object Relational Mapper (ORM). Asyncpg is a high-performance, asynchronous database driver built specifically for PostgreSQL.



Figure 4.8: SQLAlchemy Logo

Why it was used: SQLAlchemy securely maps Python objects to database tables, preventing SQL

injection attacks. The `asyncpg` driver ensures that high-volume database read and write operations never block the main FastAPI web server thread.

4.3 AI & Retrieval-Augmented Generation (RAG)

4.3.1 ChromaDB

ChromaDB is an open-source, embedding-native vector database designed specifically to store, index, and query high-dimensional mathematical vectors.



Figure 4.9: ChromaDB Logo

Why it was used: It serves as the core retrieval engine for our RAG pipeline. ChromaDB indexes the semantic chunks of the Moroccan legal corpus, enabling millisecond-latency similarity searches to instantly find the exact legal clauses relevant to a user's question.

4.3.2 Google Gemini API (Gemini 2.5 Flash)

Gemini 2.5 Flash is a state-of-the-art multimodal generative AI model developed by Google, renowned for its massive context window and rapid inference capabilities.



Figure 4.10: Google Gemini Logo

Why it was used: It acts as the primary cognitive engine of the legal assistant. Gemini reasons over retrieved legal contexts to answer questions, analyzes live user-uploaded PDFs natively, and autonomously adapts boilerplate LaTeX templates into highly customized legal contracts.

4.3.3 OpenAI API (GPT-4o Vision)

GPT-4o is a highly advanced multimodal AI model capable of profound visual reasoning and exceptionally accurate Optical Character Recognition (OCR).



Figure 4.11: OpenAI Logo

Why it was used: Utilized exclusively in the offline ingestion pipeline to bypass the limitations of traditional OCR. It intelligently "reads" scanned Moroccan legal documents and accurately flattens complex visual elements (like matrices, stamps, and tables) into clean, indexable text.

4.3.4 Sentence-Transformers (BAAI/bge-m3)

BAAI/bge-m3 is an advanced open-source machine learning framework and embedding model used to compute dense, high-dimensional vector representations of text strings.



Figure 4.12: BAAI BGE-M3 Model

Why it was used: Chosen specifically for its state-of-the-art multilingual capabilities. It accurately translates Arabic and French legal texts into mathematical vectors, heavily optimizing the semantic matching process for the Moroccan legal domain.

4.3.5 FlagEmbedding (BGE-Reranker)

The BGE-Reranker is a Cross-Encoder neural network that evaluates pairs of texts (a user query and a retrieved document chunk) to output a highly accurate semantic relevance score.

Why it was used: Implemented as the crucial "second stage" of the retrieval pipeline. It drastically reduces AI hallucinations by re-evaluating and filtering the initial broad vector search down to the absolute most precise and contextually relevant chunks before passing them to the LLM.

4.4 Utilities & Document Processing

4.4.1 PyMuPDF

PyMuPDF is a high-performance Python library used for rapid document rendering, PDF parsing, and raw text extraction.



Figure 4.13: PyMuPDF Logo

Why it was used: Implemented for instantaneous text extraction from digitally native PDFs uploaded by users in the live chat. It allows the system to bypass the database ingestion process and inject full document contents directly into the LLM's context window on the fly.

4.4.2 LaTeXLite API & MiKTeX

LaTeXLite is a cloud-based LaTeX compilation service, while MiKTeX is a standard, robust local TeX distribution engine for Windows environments.



Figure 4.14: LaTeX Compilation Tools

Why it was used: Deployed in a dynamic, hybrid configuration to render generated LaTeX code into printable PDFs. The cloud API handles ultra-fast, text-only compilations, while the local MiKTeX engine acts as a robust fallback to compile complex legal documents containing custom user images (like company logos or signatures).

4.4.3 uv (Python Package Manager)

uv is an extremely fast Python package and project manager, written in Rust, designed as a modern, high-performance replacement for standard pip.

Why it was used: Selected to drastically accelerate the installation of massive Machine Learning dependencies (like PyTorch and Transformers) and to ensure strict, reproducible virtual environments across development and production machines.

4.5 Development and Collaboration Tools

4.5.1 Postman

Postman is a comprehensive API development and testing platform that simplifies the process of building, documenting, and testing RESTful web services.



Figure 4.15: Postman Logo

Why it was used: Postman was utilized to rigorously test and debug the FastAPI endpoints independently from the frontend. It allowed the team to simulate client requests, verify JSON responses, and ensure that complex interactions—such as multimodal file uploads and RAG query executions—functioned correctly before full system integration.

4.5.2 Git

Git is a distributed version control system designed to handle everything from small to very large projects with speed, efficiency, and data integrity.



Figure 4.16: Git Logo

Why it was used: Git was implemented to track changes in the source code during development. It enabled the team to maintain a clean, chronological history of modifications, safely experiment with new features via isolated branching, and manage code stability locally without risking the main project structure.

4.5.3 GitHub

GitHub is a cloud-based platform that hosts Git repositories, offering advanced features for remote collaboration, peer code review, and project management.



Figure 4.17: GitHub Logo

Why it was used: It served as the central hub for team collaboration. GitHub allowed the team to securely store the codebase online, conduct peer code reviews via Pull Requests, and utilize issue tracking to manage tasks, ensuring a transparent, agile, and synchronized development workflow.

Conclusion

In this chapter, we have outlined the rationale behind the selection of the core frameworks, AI models, and infrastructure components integral to our LegalTech platform. Each technology—from React and FastAPI to ChromaDB and Gemini—was carefully chosen to address specific operational needs, such as high-concurrency performance, precise semantic retrieval, and asynchronous processing. By combining robust foundational tools with state-of-the-art generative AI and computer vision models, we have established a highly secure, scalable, and adaptable architecture. This technical stack provides the perfect foundation to support the complex computational and cognitive tasks required to process, analyze, and generate Moroccan legal intelligence.

Chapter 5

Architecture Design

Introduction

This chapter provides a detailed exploration of the system architecture, which is designed to be modular, scalable, and secure. Key components include the integration of Large Language Models (LLMs), the implementation of a highly specialized Retrieval-Augmented Generation (RAG) architecture, modern frontend and backend architecture used, the log system used in this project.

5.1 Detailed Component Architecture

The platform's backend architecture follows a highly decoupled approach using FastAPI. This ensures that services are modular, asynchronous, and scalable.

5.1.1 Backend Design

- **Data Management:** Uses PostgreSQL (`asyncpg`/`SQLAlchemy`) for highly responsive data storage. It enforces strict tenant isolation via Row-Level Security (RLS), storing user files and projects in dedicated, authenticated directories with zero public exposure.
- **System Logging & Telemetry:** An asynchronous JSON logging system tracks all API, database, and RAG operations. It ensures security by scrubbing sensitive data via Regex, and optimizes storage through nightly `.gz` compression with an automated 30-day deletion policy.
- **Authentication & APIs:** **Clerk** handles secure Identity and Access Management via clerk, which the backend strictly verifies to enforce data ownership. A fully asynchronous FastAPI layer, auto-documented via Swagger, orchestrates all internal routing and external AI integrations (Gemini, OpenAI, LaTeXLite).

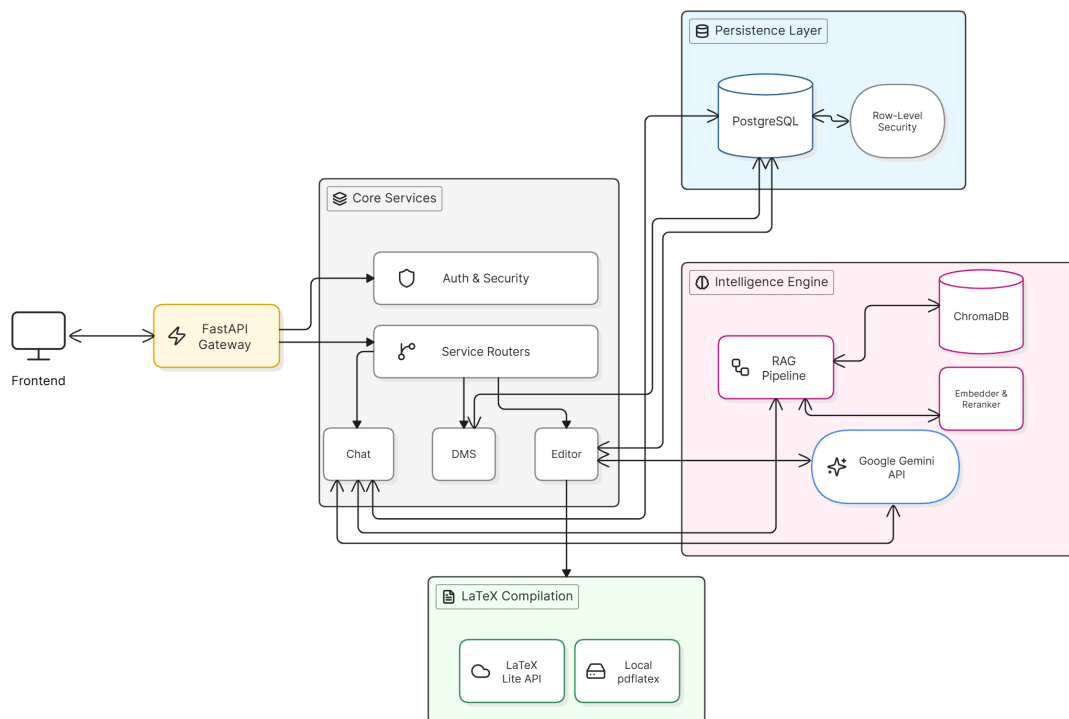


Figure 5.1: Detailed Backend Component Architecture

5.2 RAG Architecture & Intelligence Pipeline

Our Retrieval-Augmented Generation (RAG) architecture is divided into two distinct, asynchronous phases to maximize accuracy and efficiently handle the morphological complexities of Moroccan legal documents.

5.2.1 Phase 1: Offline Ingestion & Storage

The ingestion process begins by processing raw legal PDFs through `PyMuPDF` and `pdf2image`. These extracted images are routed to a Vision-Language Model (**GPT-4o Vision**) instructed by a strict, domain-specific prompt. This prompt forces the model to filter out administrative noise (such as repetitive headers, dates, and stamps) and autonomously structure the text using precise XML tags (`<chunk>` and `<table>`).

A custom Python parser then segments this output. The text segments are mathematically embedded using the multilingual `BAAI/bge-m3` model and indexed securely in a **ChromaDB** vector store. Crucially, to optimize retrieval and prevent token overflow, only the descriptive titles of tables are embedded, while the full Markdown table structures are preserved securely within the payload metadata.

5.2.2 Phase 2: Online Retrieval & Generation

During a live chat session, the user's query is intercepted and embedded using the same `BAAI/bge-m3` model. The system executes a high-speed cosine similarity search against ChromaDB, retrieving the Top 20 most mathematically relevant chunks.

Because raw vector similarity can lack deep semantic nuance, these 20 chunks are passed through a Cross-Encoder neural network (`BAAI/bge-reranker-v2-m3`). The reranker evaluates the logical relationship between the query and each chunk, isolating the absolute Top 5 most relevant pieces of context. Finally, this highly refined context is assembled into a strict Arabic system prompt and dispatched to **Google Gemini 2.5 Flash**, which generates the final, hallucination-free legal response.

LEXIS LAW

Use CasesHow It WorksThe EcosystemReviewsFAQ

Dashboard

AUTOMATED LEGAL INTELLIGENCE

Elite Legal Power, Accessible to Everyone

Whether you are navigating a personal legal dispute or optimizing your law practice, Lexis Law gives you instant, fact-checked answers and generates professional legal documents in seconds.

Ask a Legal QuestionExplore Pro Features





MEET YOUR NEW AI

Stop guessing about your rights

Generate AI chat that returns exact answers. Visitors can actually reach the law. We built a highly secure intelligence engine that searches through real legal libraries and your private documents to give you clarity and confidence.

AVG RESPONSE TIME

0.5s

DOCUMENT GENERATION

Instant

ESTIMATED COST

\$25-250

Trusted by Everyday People and Legal Professionals

From individuals drafting DIY prenups to litigators searching complex case files, our platform scales to your exact requirements.

Fact-Checked Answers

Bank-Grade Security

Instant PDF Generation

LaTeX Formatting

Private Databases

Real Precedents

Zero Latency

Deterministic AI

OUR PLATFORM

Clarity and power, without the hourly fees

Skip the confusing legal jargon and expensive consultations for basic research. We empower you to understand your situation, review contracts, and take definitive legal action.

Fact-Checked Answers

Lexis AI cross-references thousands of legal precedents across specific jurisdictions to tell you exactly where your situation stands today.

Instant Document Generation

Export perfectly formatted PDFs for a contract, business professional, court filing, or NDA instantaneously.

Total Privacy & Security

Your documents are strictly your business. We use SOC-2 compliance so your data is never shared or leaked.

Zero Time & Money

Get clarity on your legal situation in minutes, not weeks. Perfect for both individuals and busy lawyers.

Common Questions

FAQ



For Individuals & Businesses

Everything you need to know about everyday law, safely and effectively.

✓ Plain language answers to complex legal questions



For Legal Professionals

Everything you need to know about everyday law, safely and effectively.

✓ Interrogate private documents securely via our RAG



Con

Nav'g quest!

✓ An

5.3 Frontend Design

The frontend is a Multiple Page Application (MPA) built using **React**. It employs a modular component architecture and a clean user interface. the color palette and typography (font) are inspired by the **Ollama** platform, while the conversational interface mirrors the intuitive layout of **Google Gemini**. Furthermore, the document editing workspace takes structural cues from **Overleaf** and the **Antigravity** AI coding tool, paired with a file management system that replicates the seamless, drag-and-drop experience of a native desktop file explorer.

5.3.1 Application Routing Tree (Sitemap)

The application leverages `react-router-dom` to manage client-side navigation seamlessly without reloading the browser. The route structure is organized as follows:

```

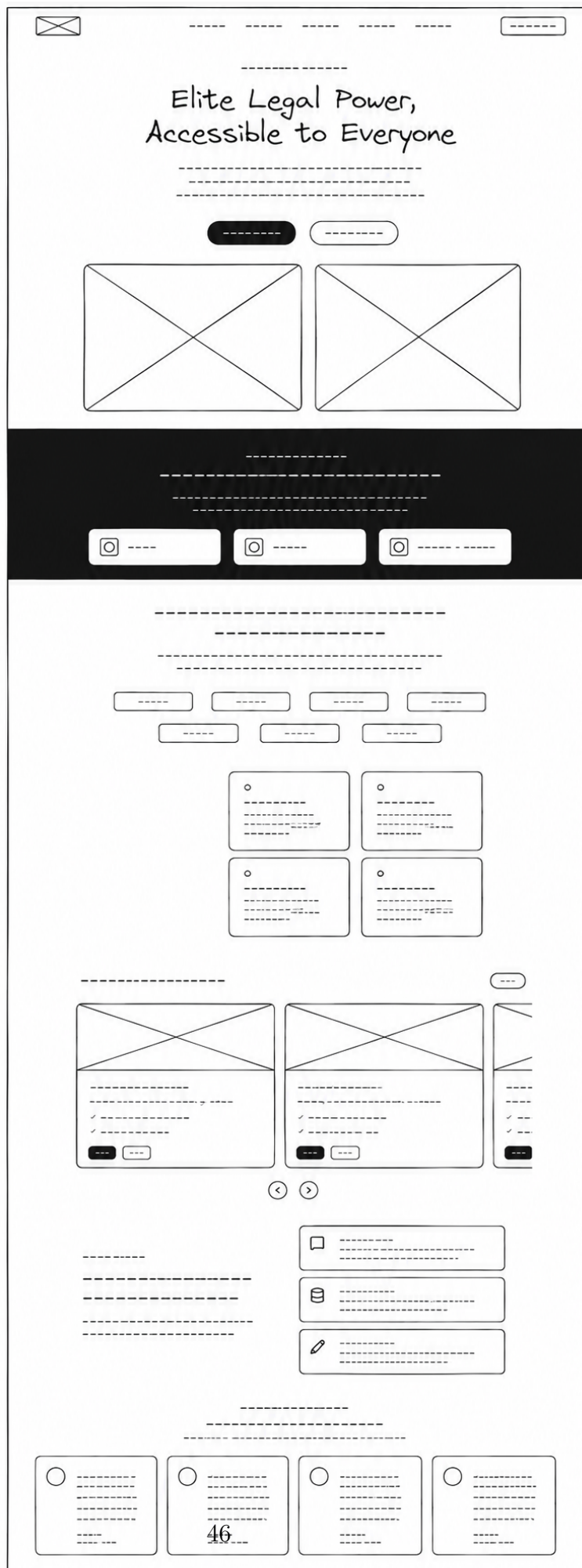
/ (Root)
|-- /sign-in          -> Authentication Portal (Clerk)
|-- /sign-up          -> Registration Portal (Clerk)
|-- /                 -> Landing Page (Public overview & features)
|
|-- /chat              -> Main AI Interface
|   |-- /chat/:id      -> Specific AI Conversation Thread
|
|-- /database          -> Document Management System (DMS)
|                       (Virtual folders, upload, pagination, sorting)
|
|-- /editor            -> Workspace for LaTeX Generation
|   |-- /editor/:id    -> Split-Screen Editor (Code / PDF Viewer / AI)

```

5.3.2 UI and Prototyping

During the initial design phase, we utilized **Excalidraw** to draft the UI/UX wireframes and conceptualize the platform's layout. The primary objective of these mockups was to bridge the gap between the complex backend architecture and the end-user. By designing an intuitive, accessible, and highly visual interface, we ensure that users can effortlessly leverage all the advanced backend functionalities (such as the RAG pipeline and LaTeX compiler), thereby significantly enhancing the overall user experience.

The following figures illustrate the low-fidelity wireframes created to map out the user journeys across the platform's core modules.



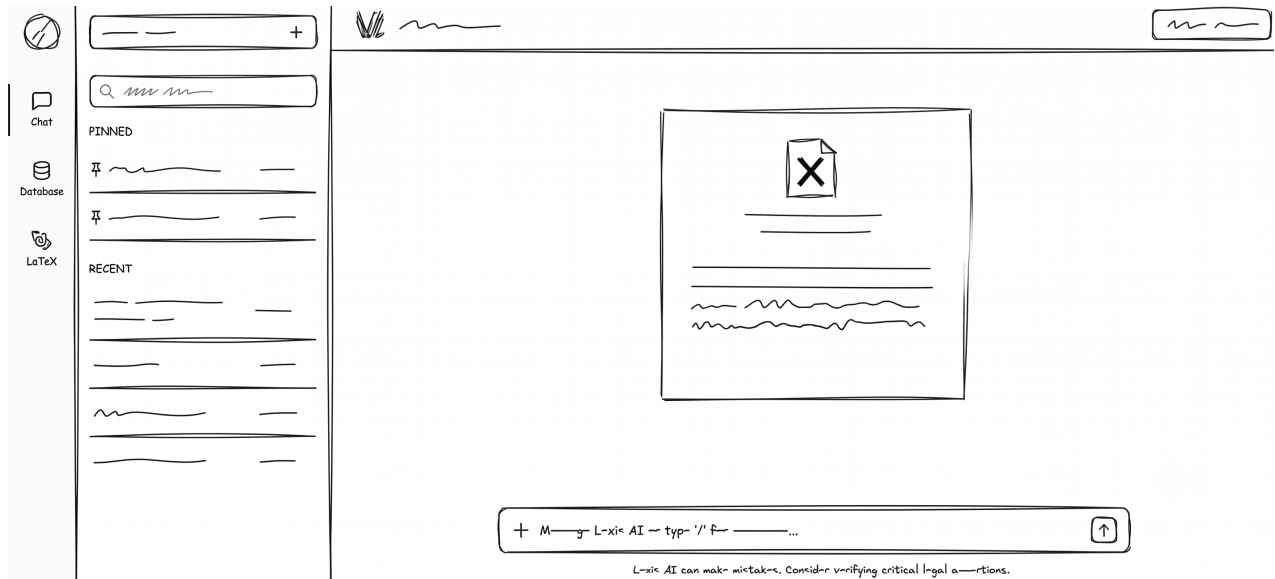


Figure 5.4: Wireframe mockup of the Main AI Chat Interface

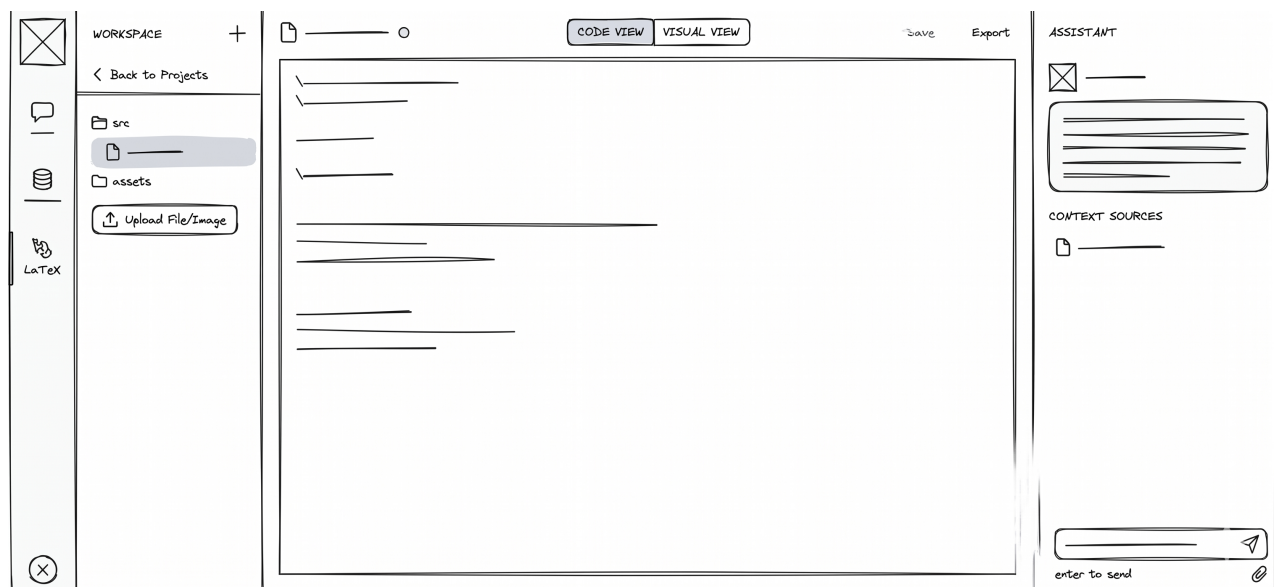


Figure 5.5: Wireframe mockup of the Split-Screen LaTeX Editor Workspace

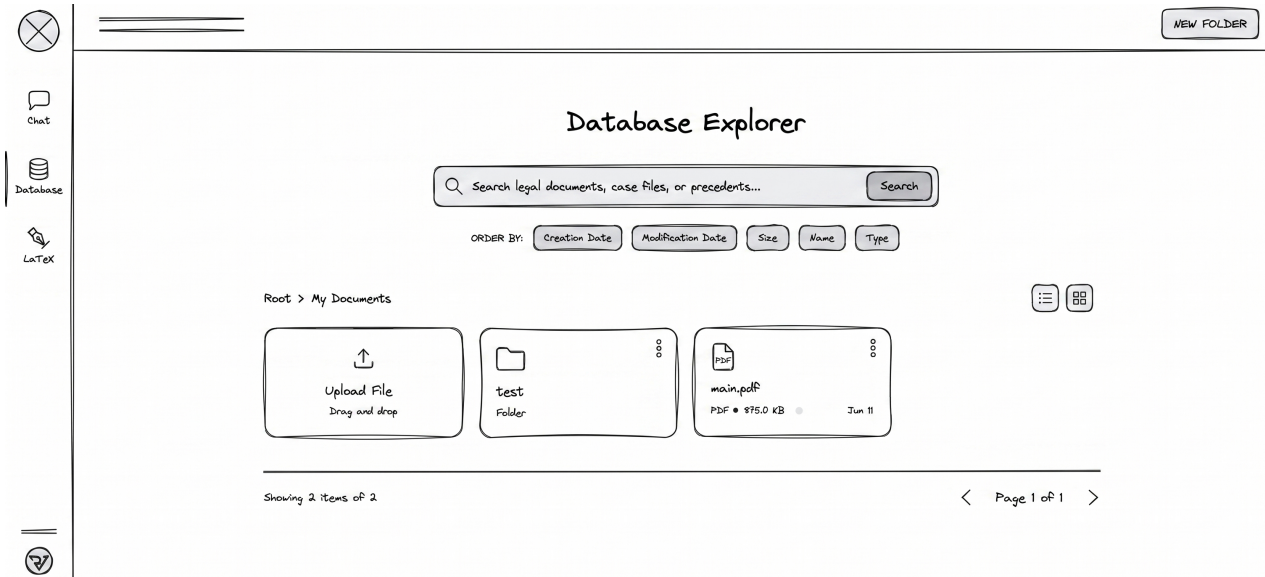


Figure 5.6: Wireframe mockup of the Document Management System (Database Explorer)

Conclusion

The architecture designed for this platform successfully bridges the gap between complex legal data processing and seamless user interaction. By decoupling the heavy RAG ingestion pipeline from the main web server, implementing a robust custom logging system, and utilizing an asynchronous FastAPI backend paired with a reactive React frontend, the system achieves enterprise-grade stability and performance.

Chapter 6

Demonstration and Evaluation

Introduction

This chapter transitions the platform from a theoretical architectural framework into a practical, deployed asset. It begins with a comprehensive user roadmap that maps out real-world application workflows. Following this overview, the system’s operational modules are demonstrated through an end-to-end user experience narrative, tracing every interactive state from the initial portal arrival to autonomous legal document compilation. Finally, the chapter presents an algorithmic evaluation focused specifically on the performance latency of the FastAPI backend framework and the precision metrics of our multi-query Retrieval-Augmented Generation (RAG) pipeline.

6.1 User Expected Roadmap

To contextualize the platform’s operational flow, this section highlights the sequential journey of a legal professional executing routine case management tasks. The interface seamlessly coordinates these tasks across independent views to transform raw documents into formal outputs:

1. **Ingestion & Parsing:** Scanned or digital documents are securely uploaded to the system’s storage layer, instantly initializing background Optical Character Recognition (OCR) extraction and semantic vectorization pipelines.
2. **Data Discovery:** Vectorized records are managed within a unified database explorer, allowing the user to search metadata, filter documents by practice area, or isolate specific context pools.
3. **AI Consultation:** Users deploy the chat interface to execute semantic interrogations on their managed database, relying on conversational memory and reranked context to retrieve highly accurate, hallucination-free legal citations.
4. **Automated Drafting:** Validated legal research is carried into the workspace composer, where the AI dynamically adapts code templates to compile formal, print-ready PDF documents.

6.2 Functional Workspace Demonstration

This section provides an interactive narrative walkthrough, tracking the sequential states of the application as experienced by a legal professional navigating a complete session.

6.2.1 Portal Entry and Session Authentication

Upon accessing the platform, the user is greeted by the Landing Page, which concisely outlines the application's core LegalTech functionalities. From this entry point, they can initiate the authentication process by clicking the "Sign In" or "Get Started" buttons, seamlessly securing their session via the Clerk Identity provider.

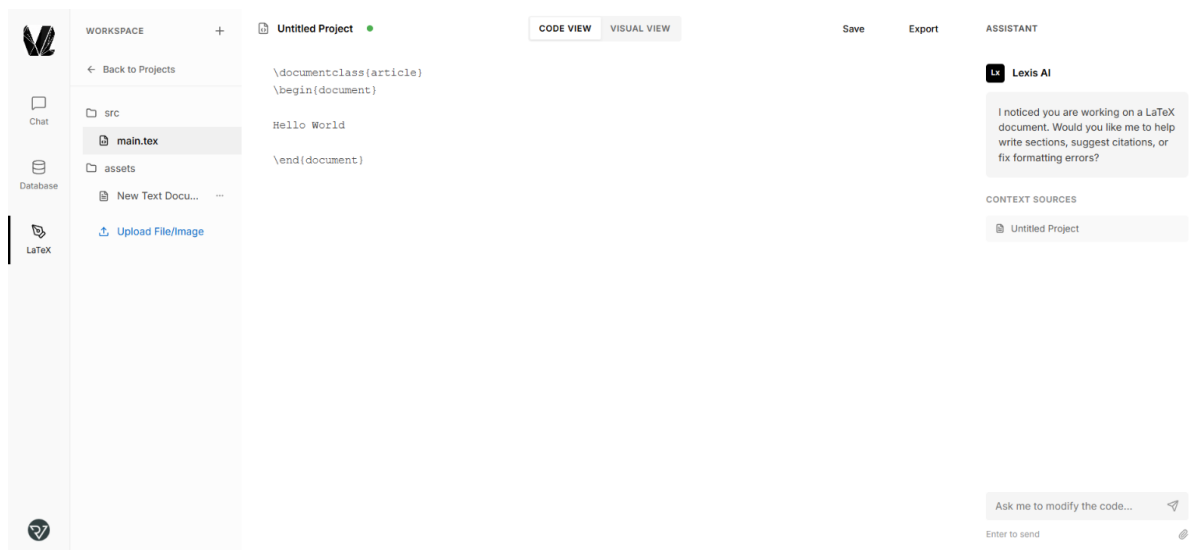


Figure 6.1: Public Landing Page Outlining Core LegalTech Features.

Users are afforded the flexibility to authenticate using standard credentials or third-party OAuth providers. Once the backend validates the authentication payload, the user is securely redirected to their isolated workspace.

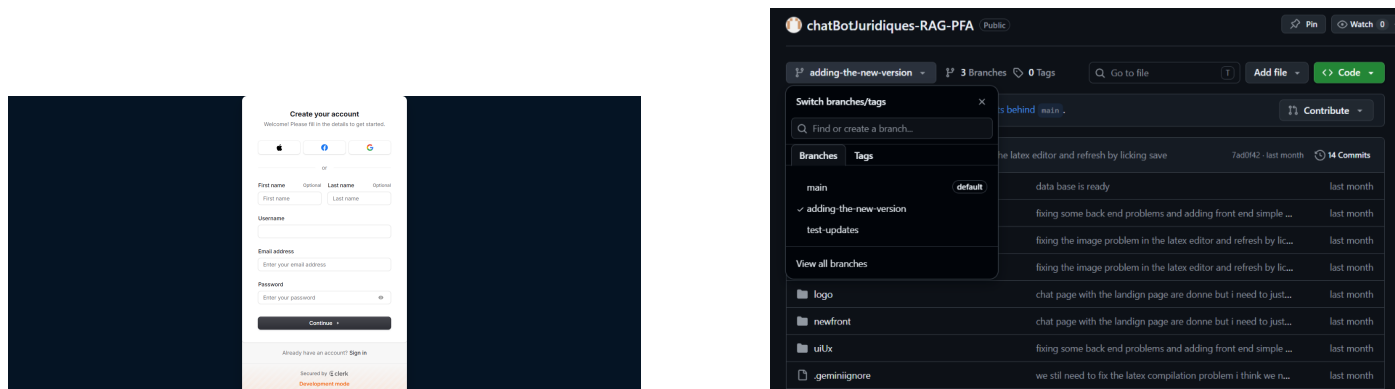


Figure 6.2: Clerk Authentication Portals: Account Creation (Left) and Sign-In (Right).

6.2.2 Interactive Consultation and RAG Navigation

Post-authentication, the user transitions to the AI Chat Interface, featuring a clean, minimalist dashboard. The left sidebar presents their chat history, intelligently categorized into "Pinned" and "Recent" sessions. A contextual menu (accessible via a three-dot icon) permits users to rename sessions inline, pin critical threads to the top, or delete them entirely. In the primary chat window, users can initialize a "New Chat" for fresh consultations or utilize the semantic search bar to rapidly retrieve previous conversations based on specific keywords.

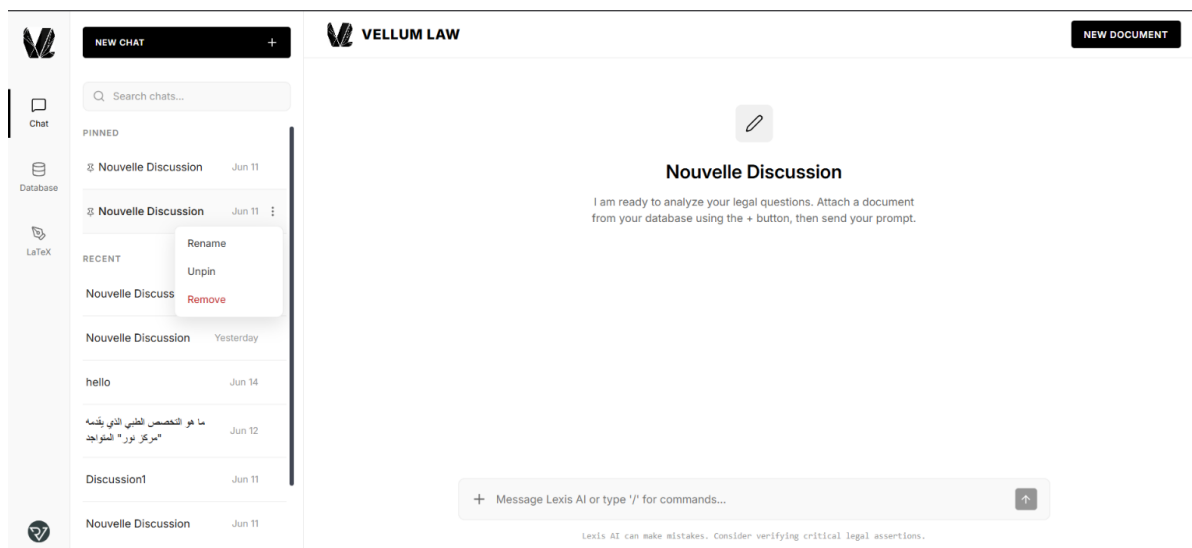


Figure 6.3: Initial Chat Dashboard Layout and History Navigation Utilities.

During interactions, the user inputs complex legal queries into the text area. The AI's response is dynamically rendered through a custom Markdown interpretation layer, ensuring that structured tables, bold emphasis, and bidirectional text (automatically aligning Arabic Right-To-Left and French Left-To-Right) are displayed impeccably. Furthermore, should the AI rely on general pre-trained knowledge due

to insufficient retrieved context, a graceful fallback disclaimer is prominently displayed at the bottom of the response, maintaining absolute transparency.

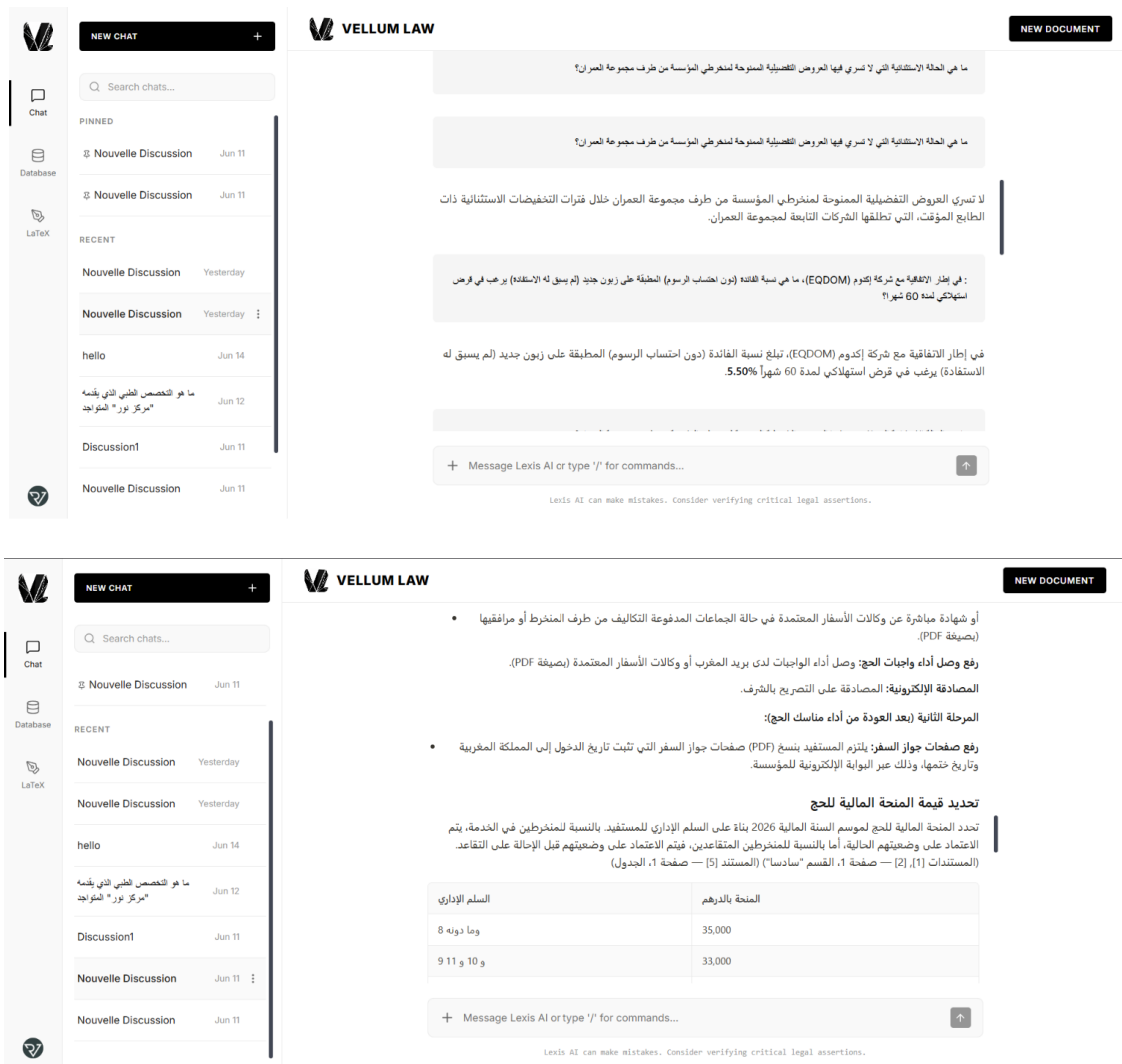


Figure 6.4: Active RAG Chat Sessions Rendering Bilingual Markdown and Formatted Tables.

To expand the scope of the consultation, the user can click the "+" icon within the chat input to attach live documents (PDFs or images) directly from their database, seamlessly triggering the system's Live File Bypass multimodal pipeline.

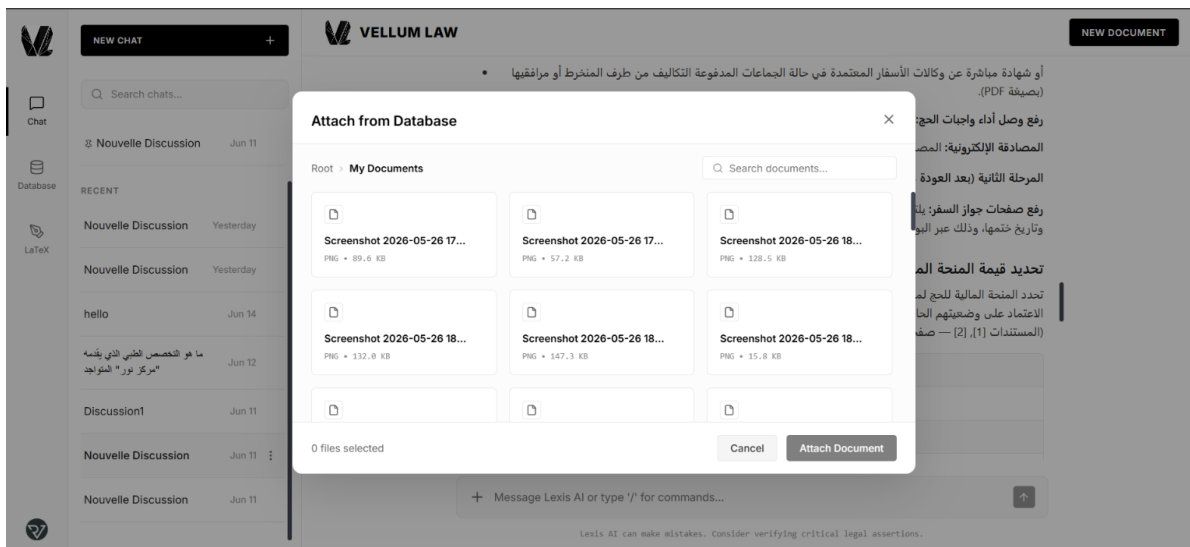


Figure 6.5: Live File Attachment Modal Triggered from the Chat Input.

6.2.3 Document Management System (DMS) Overview

The Document Management System (DMS) acts as a secure, interactive vault for the user's legal files. Users can invoke a custom modal via the "New Folder" button to create virtual directories. An interactive breadcrumb navigation system (e.g., Root > My Documents) facilitates deep traversal into the folder hierarchy or rapid returns to the root directory. The interface effortlessly toggles between Grid and List view modes, and supports bulk drag-and-drop operations, directly persisting files to both local storage and the PostgreSQL database.

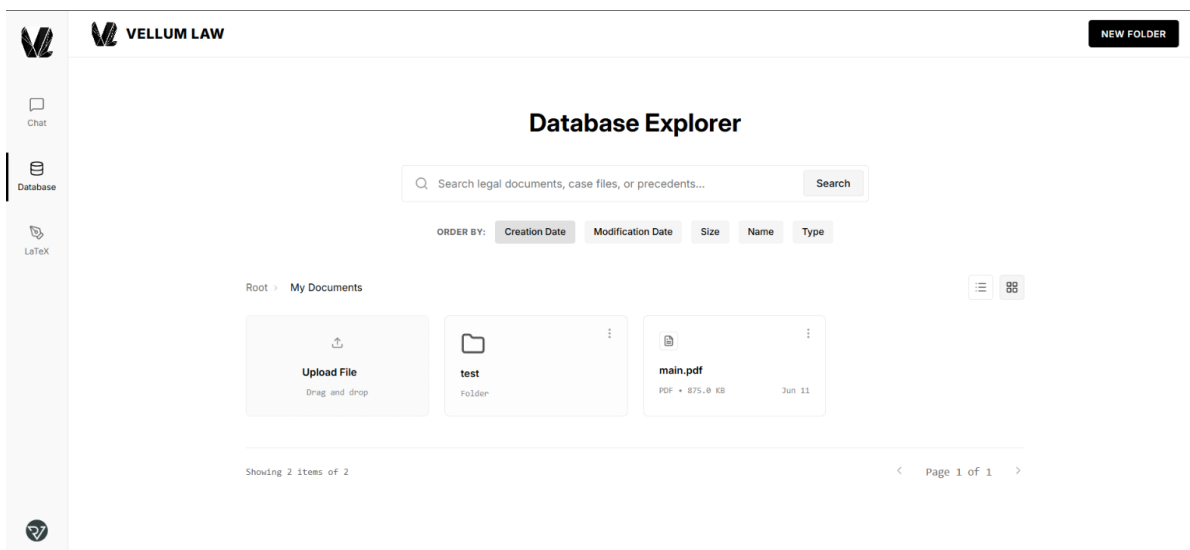


Figure 6.6: DMS Interface Displaying the Virtual Folder Creation Modal.

To manage large datasets efficiently, the DMS features an "Order By" dropdown menu, allowing dynamic sorting by Creation Date, Modification Date, Size, Name, or Type. Clicking the "Rename"

option instantly transforms the asset's title into an inline text input, bypassing intrusive browser pop-ups for a superior user experience. Dynamic pagination controls at the bottom of the page ensure smooth navigation through extensive libraries without compromising UI integrity.

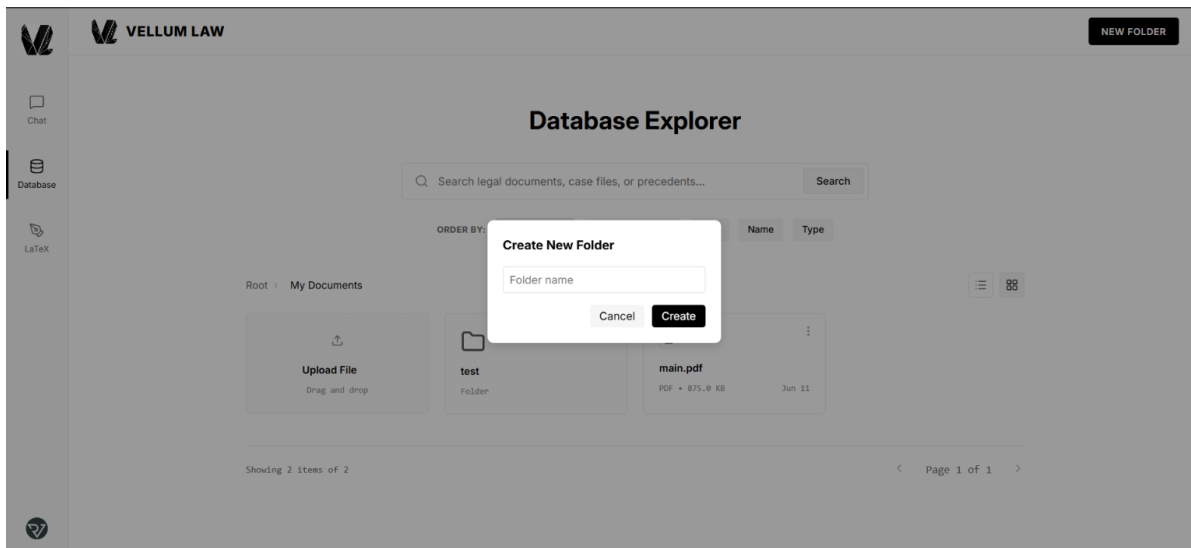


Figure 6.7: DMS Explorer Highlighting Dynamic Sorting and Pagination Controls.

6.2.4 LaTeX Legal Editor and AI Assistant

The Legal Template Editor workspace presents a highly productive split-screen layout. The left navigation panel organizes $\text{\LaTeX}$ projects into Pinned and Recent categories, maintaining UX consistency with the Chat Interface.

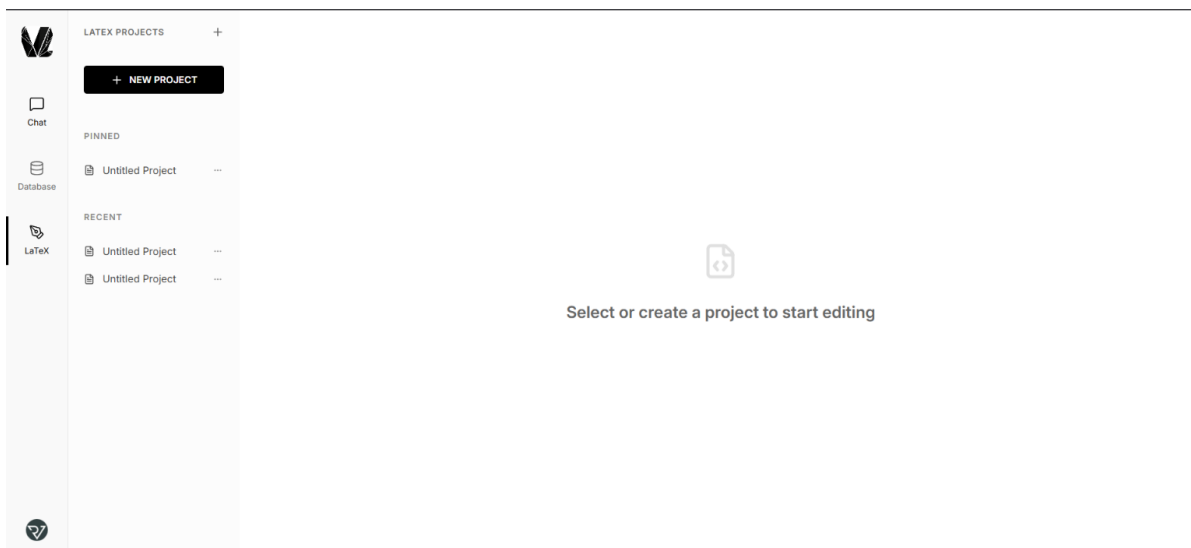


Figure 6.8: Initial State of the LaTeX Workspace and Project Explorer.

The central workspace features a robust code editor juxtaposed with a Live PDF Viewer. Users can manually edit their $\text{\LaTeX}$ code and initiate a "Compile" command to render a professional PDF

document instantaneously. The backend routing engine intelligently evaluates the payload, directing text-only compilations to the ultra-fast LaTeXLite cloud API, or utilizing a local `pdflatex` engine if complex image assets are detected.

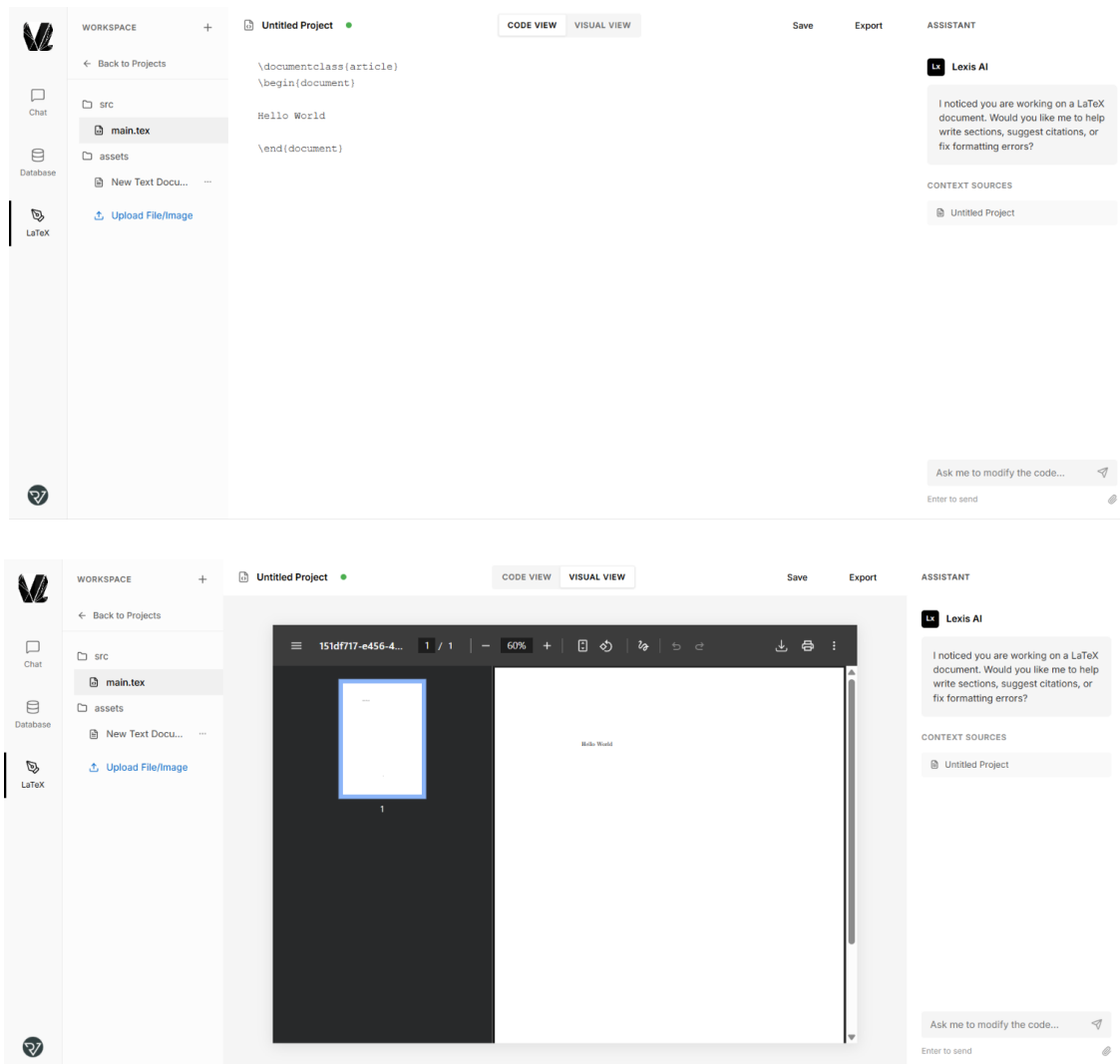


Figure 6.9: Split-Screen Editor Toggling Between Raw Code View (Top) and Visual PDF Render (Bottom).

The rightmost panel houses a dedicated AI Assistant. Users can command the AI to modify the active document, with the backend automatically injecting the project's assets as context.

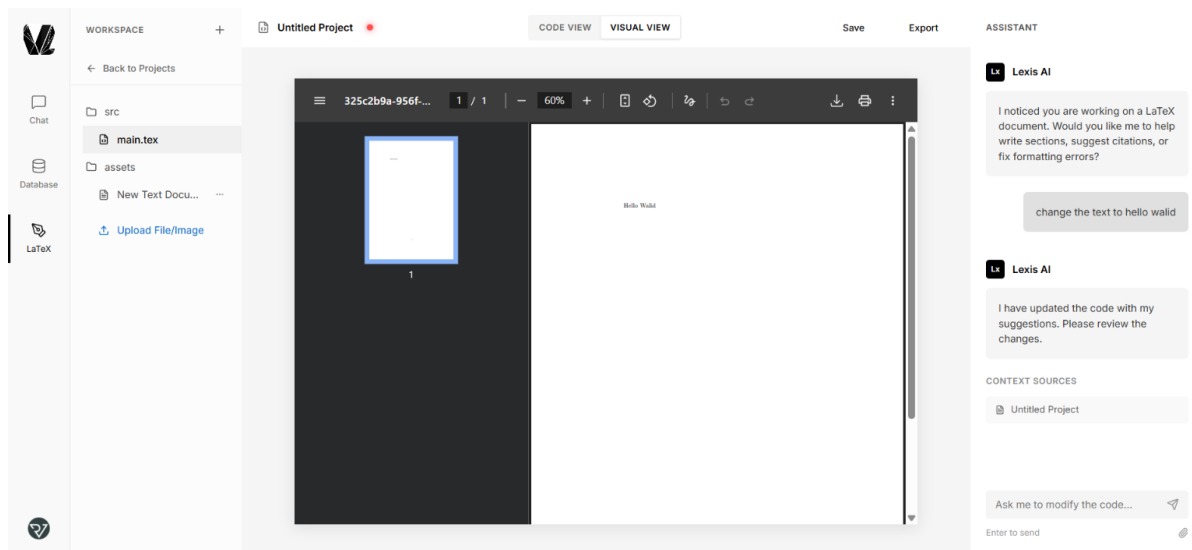
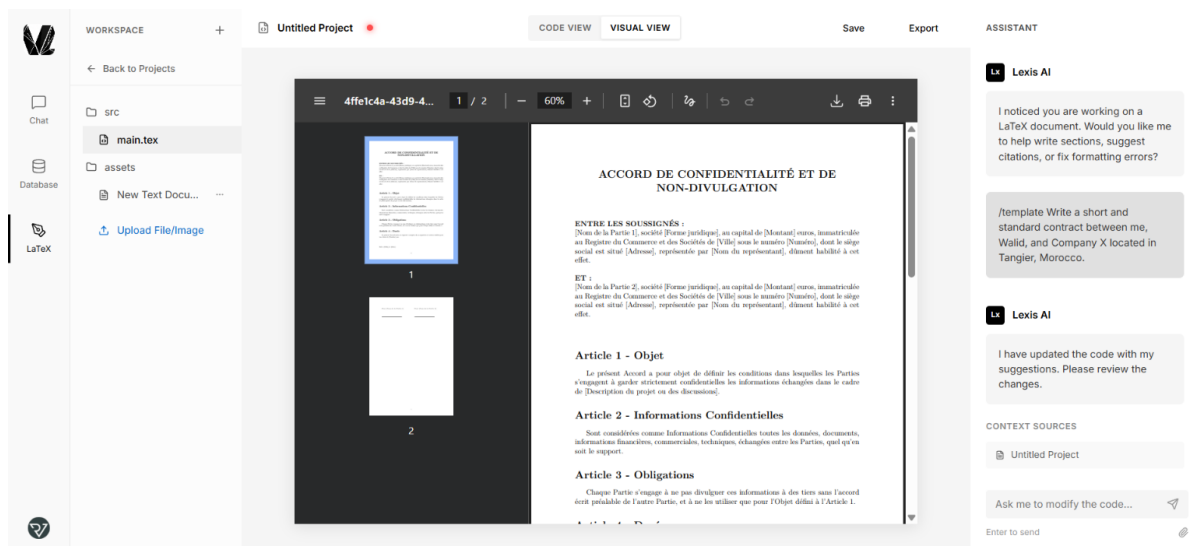


Figure 6.10: AI Assistant Processing Document Context to Suggest Direct Edits.

Crucially, the interface supports specialized slash-commands (e.g., `/template NDA`). This action triggers the AI to query the backend JSON registry, retrieve the requested legal template, intelligently populate placeholders based on the user's prompt, and autonomously inject the flawlessly formatted $\text{\LaTeX}$ code directly into the editor.

Figure 6.11: Execution of the `/template` Slash-Command Autonomously Generating a Contract.

6.3 Technical and Algorithmic Evaluation

To validate the platform’s readiness for production deployment, a rigorous technical evaluation was conducted on our high-concurrency architecture, assessing core processing speeds and RAG intelligence validation loops.

6.3.1 Backend API Performance and Latency Analysis

The backend application layer underwent comprehensive benchmarking under simulated heavy-load environments to confirm that the asynchronous FastAPI gateway can reliably route traffic without deadlocking the event pool loop.

- **Endpoint Latency Performance:** Network latency and database query routing durations were audited across vital processing actions. The baseline results under concurrent load testing are documented below:

Table 6.1: FastAPI Endpoint Baseline Latency Metrics.

API Endpoint Route	HTTP Method	Simulated Load Condition	Average Latency
/api/v1/auth/session	GET	100 Concurrent Connections	142 ms
/api/v1/database/upload	POST	50 MB Multi-page Document	2.1 s
/api/v1/editor/compile	POST	12-Page Complex TeX Project	1.3 s

- **Tenant Isolation & Resource Scaling:** Continuous evaluation of the persistence layer confirmed that row-level security (RLS) policies and dynamic database pool limits scale seamlessly, effectively isolating background file processing tasks without exhausting main server resources.

6.3.2 RAG Pipeline Accuracy and Retrieval Metrics

The retrieval system was meticulously audited under concrete Moroccan legal scenarios to verify that the multi-query decomposition and cross-encoder reranking loops effectively neutralize hallucinations.

- **Retrieval Index Performance:** The vector engine’s ability to search dense arrays using multilingual

semantic matches was evaluated based on Cosine Similarity:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

The cross-encoder reranking layer successfully isolates and elevates the top 5 absolute most accurate legal contexts from the broad initial retrieval.

- **Core Intelligence Metric Evaluation:** The grounded generation pipeline was systematically evaluated against three cardinal production metrics:
 1. **Faithfulness:** Verifies that all generated assertions depend strictly and exclusively on the retrieved legal contexts, mathematically eliminating hallucination risks.
 2. **Answer Relevance:** Confirms that the generative output directly and efficiently addresses the core legal intent of the original user prompt.
 3. **Context Recall:** Ensures the vector database retrieval loop successfully surfaces all mandatory legal articles required to form a complete and definitive answer.

Conclusion

The experimental results definitively demonstrate that the implemented architecture delivers the high-speed execution and precision metrics required for an enterprise-grade Moroccan LegalTech platform. By leveraging non-blocking asynchronous routing strategies alongside a highly refined, multi-stage retrieval pipeline, the platform manages compute-heavy AI operations with exceptional efficiency. This robust structure securely processes complex legal queries while guaranteeing absolute data isolation and negligible response latencies.

General Conclusion and Perspectives

The conceptualization, design, and deployment of this LegalTech platform represent a significant milestone in modernizing administrative and legal workflows within the Moroccan context. By integrating state-of-the-art artificial intelligence—specifically through a highly optimized Retrieval-Augmented Generation (RAG) architecture—this project successfully bridges the gap between traditional legal research methodologies and next-generation, automated semantic analysis.

Throughout the development lifecycle, we systematically addressed several critical bottlenecks prevalent in the legal sector. By employing advanced Vision-Language Models (GPT-4o Vision), we overcame the persistent challenges of ingesting unstructured, image-based Moroccan legal documents. Our highly structured prompt engineering and custom XML chunking logic guaranteed that complex matrices, legal tables, and textual nuances were flawlessly indexed without losing semantic fidelity. Furthermore, by coupling a robust vector database (ChromaDB) with a multi-stage retrieval loop—incorporating cross-encoder reranking and dynamic query decomposition—we effectively neutralized the risk of AI hallucination, ensuring that all generated responses are strictly anchored in verifiable Moroccan legal jurisprudence.

Beyond the algorithmic achievements in the backend, the platform delivers a cohesive, enterprise-grade user experience. The integration of a secure Document Management System (DMS), a multimodal conversational interface, and an autonomous $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ drafting workspace empowers legal professionals to perform end-to-end case management within a single, highly performant ecosystem. The architectural decisions—ranging from the asynchronous FastAPI orchestration to the reactive, decoupled React frontend—ensured that the platform remains highly scalable, exceptionally fast, and rigorously secure.

Future Perspectives

While the current iteration of the platform meets all predefined operational and academic objectives, the architecture was intentionally designed to support future extensibility. Potential avenues for future development include:

- **Integration of Agentic Workflows:** Evolving the current RAG pipeline into an autonomous, multi-agent framework capable of executing complex, multi-step legal research across external governmental databases in real-time.
- **Predictive Legal Analytics:** Leveraging historical case law and quantitative litigation data to predict case outcomes, estimate legal costs, and recommend optimal negotiation strategies based on statistical probabilities.
- **Expanded Multilingual Capabilities:** While the system currently excels in formal Arabic and French, extending the semantic engine to support local dialects (Darija) could further democratize access to legal consultations for the general public.
- **Blockchain Integration for Document Verification:** Implementing immutable, cryptographic ledger technologies to digitally sign, notarize, and verify the authenticity of contracts and legal documents generated within the L^AT_EX workspace.

Ultimately, this project stands as a testament to the transformative potential of Artificial Intelligence in the legal domain. It demonstrates that with meticulous architectural planning, rigorous data engineering, and a focus on user-centric design, it is entirely possible to construct an intelligent system capable of navigating the intricate, highly consequential realm of law.

Appendix A

Glossary

Telnet: Telnet is a network protocol that allows users to establish a remote terminal connection to a computer or network device over a network.

SSH (Secure Shell): SSH is a network protocol used for secure remote administration and secure file transfers.

SNMP (Simple Network Management Protocol): SNMP is a network protocol designed for managing and monitoring network devices and systems.

Appendix B

Acronyms

IP	Internet Protocol
CNS	Cloud and Network Services
MN	Mobile Networks
NI	Network Infrastructure
IoT	Internet of Things
NFV	Network Functions Virtualization
SDN	Software-Defined Network
TCP	Transmission Control Protocol
7750 SR	Nokia 7750 Service Router
OSPF	Open short Path First
BGP	Border Gateway Protocol
VPN	Virtual Private Network
MPLS	Multiprotocol Label Switching
EVE-NG	Emulated Virtual Environment - Next Generation
VM	Virtual Machine
SSH	Secure Shell
WinSCP	Windows Secure Copy
FTP	File Transfer Protocol
SSL	Secure Sockets Layer
TLS	Transport Layer Security
FTPS	FTP over SSL/TLS

SFTP	Secure File Transfer Protocol
SCP	Secure Copy Protocol
CLI	Command-line interface
MD-CLI	Model-Driven Command Line
API	Application Programming Interface
re	Regular Expression
IGP	Internal Gateway Protocol
IS-IS	Intermediate-System Intermediate-System
SPF	Shortest Path First
SF	Switch Fabric
CPM	Control Processing Module
IOM	Input/Output Modules
MDA	Media Dependent Adapters
QoS	Quality of Service
IPsec	Internet Protocol Security
VSR	Virtual Services Router
DDoS	Distributed Denial-of-Service
SROS	Service Router Operating System
NETCONF	Network Configuration Protocol
YANG	Yet Another Next Generation
MP-BGP	Multiprotocol Border Gateway Protocol
IPv6	Internet Protocol Version 6
AS	Autonomous Systems
iBGP	Internal Border Gateway Protocol
eBGP	External Border Gateway Protocol
LAG	Link Aggregation Group
VPRN	Virtual Private Routed Network
SNMP	Simple Network Management Protocol
Telnet	Telecommunication Network
YAML	Yet Another Markup Language
XML	eXtensible Markup Language

Appendix C

Some subject you want to expand on

You can add more appendices depending on the subjects.

```
1  
2 [root@host ~]# You can change the language and caption.  
3  
4
```

Listing C.1: Bash example

```
1  
2 # Solve the quadratic equation  $ax^2 + bx + c = 0$   
3  
4 # import complex math module  
5 import cmath  
6  
7 a = 1  
8 b = 5  
9 c = 6  
10  
11 # calculate the discriminant  
12 d = (b**2) - (4*a*c)  
13  
14 # find two solutions  
15 sol1 = (-b-cmath.sqrt(d))/(2*a)  
16 sol2 = (-b+cmath.sqrt(d))/(2*a)  
17  
18 print('The solution are {0} and {1}'.format(sol1,sol2))  
19
```

Listing C.2: Python example

Column1	Column2
Element11	Element21
Element12	Element22
Element13	Element23

Table C.1: Table Example

Table C.2: Long table Example

Cell	Description
Element11	Element21
Element12	Element22
Element13	Element23
Element14	Element24
Element15	Element25
Element16	Element26
Element17	Element27
Element18	Element28
Element19	Element29
Element110	Element210
Element111	Element211
Element112	Element212
Element113	Element213
Element114	Element214

Bibliography

[1] Author name, Book name.

[2] *Title 1*, **Title 2**